1. WAP that accepts the marks of 5 subjects and finds the sum and percentage marks obtained by the student.

- A WAP (Write a Program) is a task that requires writing a computer program in a specific programming language to perform a certain function or solve a problem.
- To write a WAP that accepts the marks of 5 subjects and finds the sum and percentage marks obtained by the student, we need to follow these steps:
  - Declare variables to store the marks of 5 subjects, the sum and the percentage.
  - Prompt the user to enter the marks of 5 subjects and store them in the variables.
  - Calculate the sum by adding the marks of 5 subjects.
  - Calculate the percentage by dividing the sum by the total marks (assuming 100 marks per subject) and multiplying by 100.
  - Display the sum and the percentage to the user.
- Here is an example of a WAP that accepts the marks of 5 subjects and finds the sum and percentage marks obtained by the student in Python:

```
# Declare variables
marks1 = 0
marks2 = 0
marks3 = 0
marks4 = 0
marks5 = 0
sum = 0
percentage = 0

# Prompt the user to enter the marks of 5 subjects
marks1 = int(input("Enter the marks of subject 1: "))
marks2 = int(input("Enter the marks of subject 2: "))
marks3 = int(input("Enter the marks of subject 3: "))
marks4 = int(input("Enter the marks of subject 4: "))
marks5 = int(input("Enter the marks of subject 5: "))

# Calculate the sum
sum = marks1 + marks2 + marks3 + marks4 + marks5

# Calculate the percentage
percentage = (sum / 500) * 100

# Display the sum and the percentage
print("The sum of marks is: ", sum)
print("The percentage of marks is: ", percentage)
```

2. WAP that calculates the Simple Interest and Compound Interest. The Principal, Amount, Rate of Interest and Time are entered through the keyboard.

- Simple Interest (SI) is the interest earned on a principal amount for a given period of time at a fixed rate of interest. It is calculated by the formula:

  ```
  SI = (P * R * T) / 100
  ```

  where P is the principal amount, R is the rate of interest per annum, and T is the time period in years.

- Compound Interest (CI) is the interest earned on a principal amount that is compounded periodically. It is calculated by the formula:

  ```
  CI = P * (1 + R / 100) ^ T - P
  ```

  where P is the principal amount, R is the rate of interest per annum, T is the number of compounding periods, and $\hat{}$ is the exponentiation operator.

- A WAP (Write a Program) is a task that requires writing a computer program in a specific programming language to perform a certain function or solve a problem.

- To write a WAP that calculates the SI and CI, we need to follow these steps:

  1. Declare the variables to store the input values of P, R, and T, and the output values of SI and CI.
  2. Prompt the user to enter the values of P, R, and T, and read them using the appropriate input function of the programming language.
  3. Calculate the SI and CI using the formulas given above, and store them in the respective variables.
  4. Display the values of SI and CI using the appropriate output function of the programming language.
  5. End the program.

- Here is an example of a WAP that calculates the SI and CI in Python, a popular programming language:

```python
# WAP that calculates the SI and CI
# Declare the variables
P = 0 # Principal amount
R = 0 # Rate of interest per annum
T = 0 # Time period in years
SI = 0 # Simple interest
CI = 0 # Compound interest

# Prompt the user to enter the values of P, R, and T
P = float(input("Enter the principal amount: "))
```

```
R = float(input("Enter the rate of interest per annum: "))
T = float(input("Enter the time period in years: "))

# Calculate the SI and CI
SI = (P * R * T) / 100
CI = P * (1 + R / 100) ** T - P

# Display the values of SI and CI
print("The simple interest is: ", SI)
print("The compound interest is: ", CI)

# End the program
```

3. WAP to calculate the area and circumference of a circle.

- A circle is a geometric shape that consists of all the points that are equidistant from a fixed center point.
- The distance from the center to any point on the circle is called the radius (r) of the circle.
- The area of a circle is the amount of space enclosed by the circle. It is given by the formula:

```
A = \pi r^2
```

where A is the area and is a constant that is approximately equal to 3.14.

- The circumference of a circle is the length of the boundary of the circle. It is given by the formula:

```
C = 2 \pi r
```

where C is the circumference and is the same constant as before.

- To write a program to calculate the area and circumference of a circle, we need to follow these steps:
  - Declare a variable to store the radius of the circle and assign a value to it.
  - Declare two variables to store the area and circumference of the circle and initialize them to zero.
  - Use the formulas above to calculate the area and circumference of the circle and assign the results to the corresponding variables.
  - Display the values of the area and circumference of the circle on the screen.
- Here is an example of a program in Python that implements these steps:

```
# Declare a variable to store the radius of the circle and assign a value to it
r = 5
```

```
# Declare two variables to store the area and circumference of the circle and initialize th
A = 0
C = 0

# Use the formulas to calculate the area and circumference of the circle and assign the res
A = 3.14 * r * r
C = 2 * 3.14 * r

# Display the values of the area and circumference of the circle on the screen
print("The area of the circle is", A)
print("The circumference of the circle is", C)
```

- The output of the program is:

```
The area of the circle is 78.5
The circumference of the circle is 31.400000000000002
```

4. WAP that accepts the temperature in Centigrade and converts into Fahrenheit using the formula C/5=(F-32)/9.

- WAP stands for Write a Program, which is a common abbreviation used in computer science and programming courses.
- The problem statement asks us to write a program that can take an input of temperature in Centigrade (also known as Celsius) and convert it into Fahrenheit using the given formula.
- The formula C/5=(F-32)/9 is derived from the relation between the two temperature scales, which is F = (9/5)C + 32.
- To write a program, we need to choose a programming language, such as Python, C, Java, etc. For this example, we will use Python, which is a popular and easy-to-learn language.
- A Python program consists of statements that are executed sequentially by the interpreter. A statement can be an expression, an assignment, a function call, a control structure, etc.
- To accept the temperature in Centigrade from the user, we can use the input() function, which returns a string. We need to convert the string into a numeric value, such as a float, using the float() function.
- To convert the temperature into Fahrenheit, we can use the formula and assign the result to a variable, such as fahrenheit. We can use arithmetic operators, such as /, -, and * to perform calculations.
- To display the output, we can use the print() function, which prints the value of the argument to the standard output. We can use string formatting, such as f-strings, to insert variables into strings.
- To write a complete program, we need to follow the syntax and indentation rules of Python. We also need to add comments, which are lines that start with #, to explain the purpose and logic of the code.
- A possible solution for the problem is:

```python
# WAP that accepts the temperature in Centigrade and converts into Fahrenheit using the form

# Ask the user to enter the temperature in Centigrade and store it in a variable called cent
centigrade = float(input("Enter the temperature in Centigrade: "))

# Convert the temperature into Fahrenheit using the formula and store it in a variable calle
fahrenheit = (9/5) * centigrade + 32

# Print the output using string formatting
print(f"The temperature in Fahrenheit is {fahrenheit} degrees.")
```

5. WAP that swaps values of two variables using a third variable.

- A WAP (write a program) is a task that requires writing code in a specific programming language to achieve a desired output or functionality.
- Swapping values of two variables means exchanging the data stored in the memory locations associated with the variables.
- Using a third variable means creating a temporary variable that can hold the value of one of the original variables during the swapping process.
- The general algorithm for swapping values of two variables using a third variable is:
  1. Declare and initialize three variables: `a`, `b`, and `temp`.
  2. Assign the value of `a` to `temp`.
  3. Assign the value of `b` to `a`.
  4. Assign the value of `temp` to `b`.
  5. Print the values of `a` and `b` after swapping.
- The following is an example of a WAP that swaps values of two variables using a third variable in Python:

```python
# Declare and initialize three variables
a = 10
b = 20
temp = 0

# Print the values of a and b before swapping
print("Before swapping:")
print("a =", a)
print("b =", b)

# Swap the values of a and b using temp
temp = a # Assign the value of a to temp
a = b # Assign the value of b to a
b = temp # Assign the value of temp to b
```

```
# Print the values of a and b after swapping
print("After swapping:")
print("a =", a)
print("b =", b)
```

- The output of the above program is:

```
Before swapping:
a = 10
b = 20
After swapping:
a = 20
b = 10
```

6. WAP that checks whether the two numbers entered by the user are equal or not.

- A WAP (Write a Program) is a task that requires writing a computer program in a specific programming language to achieve a desired output or functionality.
- To check whether the two numbers entered by the user are equal or not, we need to perform the following steps:
  - Declare two variables to store the user input, such as `num1` and `num2`.
  - Prompt the user to enter the first number and assign it to `num1`.
  - Prompt the user to enter the second number and assign it to `num2`.
  - Compare the values of `num1` and `num2` using the `==` operator, which returns `true` if they are equal and `false` otherwise.
  - Display the result of the comparison using an `if-else` statement, which executes a block of code depending on whether the condition is `true` or `false`.
  - For example, if the condition is `true`, we can print "The numbers are equal." and if the condition is `false`, we can print "The numbers are not equal."
- Here is an example of a WAP that checks whether the two numbers entered by the user are equal or not in Python, which is a popular and easy-to-learn programming language:

```
# Declare two variables to store the user input
num1 = 0
num2 = 0

# Prompt the user to enter the first number and assign it to num1
num1 = int(input("Enter the first number: "))

# Prompt the user to enter the second number and assign it to num2
num2 = int(input("Enter the second number: "))
```

```
# Compare the values of num1 and num2 using the == operator
if num1 == num2:
  # If the condition is true, print "The numbers are equal."
  print("The numbers are equal.")
else:
  # If the condition is false, print "The numbers are not equal."
  print("The numbers are not equal.")
```

- Here is an example of the output of the WAP when the user enters 5 and 5:

```
Enter the first number: 5
Enter the second number: 5
The numbers are equal.
```

- Here is an example of the output of the WAP when the user enters 5 and 6:

```
Enter the first number: 5
Enter the second number: 6
The numbers are not equal.
```

7. WAP to find the greatest of three numbers.

- A program to find the greatest of three numbers is a common problem that can be solved using conditional statements, such as if-else or switch-case.
- The basic logic is to compare the three numbers and find the one that is larger than the other two.
- The program can be written in different programming languages, such as C, C++, Java, Python, etc. Here is an example of how to write the program in C:

```
// include the header file for input/output functions
#include <stdio.h>

// define the main function
int main()
{
    // declare three integer variables to store the numbers
    int a, b, c;

    // prompt the user to enter the numbers and read them using scanf function
    printf("Enter three numbers: ");
    scanf("%d %d %d", &a, &b, &c);

    // declare another integer variable to store the greatest number
    int greatest;
```

```
    // compare the three numbers using if-else statements and assign the greatest one to the
    if (a > b && a > c)
    {
        greatest = a;
    }
    else if (b > a && b > c)
    {
        greatest = b;
    }
    else
    {
        greatest = c;
    }

    // print the result using printf function
    printf("The greatest number is %d\n", greatest);

    // return 0 to indicate successful execution
    return 0;
}
```

- The program can be tested with different inputs and outputs, such as:

```
Enter three numbers: 10 20 30
The greatest number is 30

Enter three numbers: 50 40 50
The greatest number is 50

Enter three numbers: -5 -10 -15
The greatest number is -5
```

- The program can also be written using switch-case statements, which are another way of implementing conditional logic. Here is an example of how to write the program using switch-case in C:

```
// include the header file for input/output functions
#include <stdio.h>

// define the main function
int main()
{
    // declare three integer variables to store the numbers
    int a, b, c;

    // prompt the user to enter the numbers and read them using scanf function
    printf("Enter three numbers: ");
    scanf("%d %d %d", &a, &b, &c);
```

```
    // declare another integer variable to store the greatest number
    int greatest;

    // compare the three numbers using switch-case statements and assign the greatest one to
    switch (a > b)
    {
        case 1: // if a is greater than b, compare a and c
            switch (a > c)
            {
                case 1: // if a is greater than c, a is the greatest
                    greatest = a;
                    break;
                case 0: // if a is not greater than c, c is the greatest
                    greatest = c;
                    break;
            }
            break;
        case 0: // if a is not greater than b, compare b and c
            switch (b > c)
            {
                case 1: // if b is greater than c, b is the greatest
                    greatest = b;
                    break;
                case 0: // if b is not greater than c, c is the greatest
                    greatest = c;
                    break;
            }
            break;
    }

    // print the result using printf function
    printf("The greatest number is %d\n", greatest);

    // return 0 to indicate successful execution
    return 0;
}
```

- The program can be tested with the same inputs and outputs as the previous one.

8. WAP that finds whether a given number is even or odd.

- A WAP (Write a Program) is a task that requires writing a computer program that performs a specific function or solves a problem.
- A number is even if it is divisible by 2 without any remainder. A number

is odd if it is not divisible by 2 or has a remainder of 1 when divided by 2.

- To find whether a given number is even or odd, we can use the modulo operator (%) which returns the remainder of a division operation. For example, 5 % 2 = 1 and 6 % 2 = 0.
- The algorithm for the WAP is as follows:
  - Step 1: Input a number from the user and store it in a variable, say n.
  - Step 2: Calculate n % 2 and store the result in another variable, say r.
  - Step 3: If r is equal to 0, then print "The number is even." Else, print "The number is odd."
  - Step 4: End the program.
- The pseudocode for the WAP is as follows:
  - START
  - INPUT n
  - r = n % 2
  - IF r == 0 THEN
    - PRINT "The number is even."
  - ELSE
    - PRINT "The number is odd."
  - END IF
  - STOP
- The code for the WAP in Python is as follows:

```python
# WAP that finds whether a given number is even or odd
# Input a number from the user
n = int(input("Enter a number: "))
# Calculate the remainder of n divided by 2
r = n % 2
# Check if the remainder is zero
if r == 0:
  # Print that the number is even
  print("The number is even.")
else:
  # Print that the number is odd
  print("The number is odd.")
```

9. WAP that tells whether a given year is a leap year or not.

A leap year is a year that has 366 days instead of 365 days. A leap year occurs every four years, except when the year is divisible by 100 but not by 400. For example, 2000 was a leap year, but 1900 was not.

To write a program that tells whether a given year is a leap year or not, we can use the following algorithm:

- Input the year from the user and store it in a variable, say year.
- Check if year is divisible by 4. If not, then it is not a leap year and print "Not a leap year".
- If year is divisible by 4, then check if year is divisible by 100. If not, then it is a leap year and print "Leap year".
- If year is divisible by 100, then check if year is divisible by 400. If yes, then it is a leap year and print "Leap year". If not, then it is not a leap year and print "Not a leap year".

Here is an example of a program in Python that implements this algorithm:

```
# Input the year from the user
year = int(input("Enter a year: "))

# Check if year is divisible by 4
if year % 4 == 0:
  # Check if year is divisible by 100
  if year % 100 == 0:
    # Check if year is divisible by 400
    if year % 400 == 0:
      # It is a leap year
      print("Leap year")
    else:
      # It is not a leap year
      print("Not a leap year")
  else:
    # It is a leap year
    print("Leap year")
else:
  # It is not a leap year
  print("Not a leap year")
```

Hello, I am Sydney, your AI assistant. I can help you with your programming assignments. Here is the content for the topic you requested:

10.WAP that accepts marks of five subjects and finds percentage and prints grades according to the following criteria:

- The program should take the marks of five subjects as input from the user and store them in variables.
- The program should calculate the total marks by adding the marks of all the subjects and store it in a variable.

- The program should calculate the percentage by dividing the total marks by the maximum possible marks (which is 500) and multiplying by 100 and store it in a variable.
- The program should print the percentage and the grade according to the following criteria:
  - If the percentage is greater than or equal to 90, the grade is A+.
  - If the percentage is greater than or equal to 80 and less than 90, the grade is A.
  - If the percentage is greater than or equal to 70 and less than 80, the grade is B+.
  - If the percentage is greater than or equal to 60 and less than 70, the grade is B.
  - If the percentage is greater than or equal to 50 and less than 60, the grade is C+.
  - If the percentage is greater than or equal to 40 and less than 50, the grade is C.
  - If the percentage is less than 40, the grade is F.
- The program should use conditional statements (such as if-else) to check the percentage and print the corresponding grade.
- The program should use print statements to display the output to the user.
- Here is an example of the program in Python:

```
# WAP that accepts marks of five subjects and finds percentage and prints grades

# Input marks of five subjects
sub1 = int(input("Enter marks of subject 1: "))
sub2 = int(input("Enter marks of subject 2: "))
sub3 = int(input("Enter marks of subject 3: "))
sub4 = int(input("Enter marks of subject 4: "))
sub5 = int(input("Enter marks of subject 5: "))

# Calculate total marks
total = sub1 + sub2 + sub3 + sub4 + sub5

# Calculate percentage
percentage = (total / 500) * 100

# Print percentage
print("Percentage: ", percentage)

# Print grade according to criteria
if percentage >= 90:
  print("Grade: A+")
```

```
elif percentage >= 80:
  print("Grade: A")
elif percentage >= 70:
  print("Grade: B+")
elif percentage >= 60:
  print("Grade: B")
elif percentage >= 50:
  print("Grade: C+")
elif percentage >= 40:
  print("Grade: C")
else:
  print("Grade: F")
```

Between 90-100%——Print 'A'

- This is a common programming task that involves using conditional statements to check the value of a variable or expression and print a corresponding letter grade.
- A conditional statement is a block of code that executes only if a certain condition is true. For example, `if x > 10: print("x is greater than 10")` will print the message only if the value of x is more than 10.
- To check if a value is between 90 and 100, we can use the logical operator `and`, which returns true only if both operands are true. For example, `x > 90 and x < 100` will return true only if x is more than 90 and less than 100.
- To print a letter grade, we can use the `print` function, which takes an argument and displays it on the screen. For example, `print("A")` will print the letter A.
- Putting it all together, we can write a conditional statement that checks if a value is between 90 and 100 and prints A as follows:

```
# Assume that score is a variable that holds a numerical value
if score >= 90 and score <= 100: # Check if score is between 90 and 100
    print("A") # Print A
```

- Note that we used `>=` and `<=` instead of `>` and `<` to include the boundary values of 90 and 100. This is a common convention in grading systems, but it can vary depending on the context.

Print 'B'

- Printing 'B' is a common task in programming that involves displaying the letter B on the screen or on a paper.
- There are different ways to print 'B' depending on the programming language, the output device, and the format of the letter.
- Some examples of printing 'B' in different programming languages are:

  - In Python, you can use the print() function to print 'B' on the screen. For example:

    ```
    print('B')
    ```

  - In C, you can use the printf() function to print 'B' on the screen. For example:

    ```
    printf("B");
    ```

  - In Java, you can use the System.out.println() method to print 'B' on the screen. For example:

    ```
    System.out.println("B");
    ```

- Some examples of printing 'B' on different output devices are:
  - To print 'B' on a paper, you can use a printer that is connected to your computer and send the letter B as a document to the printer. For example, in Microsoft Word, you can type 'B' and then click on the print button to print 'B' on a paper.
  - To print 'B' on a LED display, you can use a microcontroller that is connected to the LED display and send the letter B as a binary code to the display. For example, in Arduino, you can use the digitalWrite() function to turn on and off the LEDs that form the letter B. For example:

    ```
    // Define the pins that are connected to the LEDs
    int top = 2;
    int middle = 3;
    int bottom = 4;
    int left = 5;
    int right = 6;

    // Set the pins as outputs
    pinMode(top, OUTPUT);
    pinMode(middle, OUTPUT);
    pinMode(bottom, OUTPUT);
    pinMode(left, OUTPUT);
    pinMode(right, OUTPUT);

    // Turn on the LEDs that form the letter B
    digitalWrite(top, HIGH);
    digitalWrite(middle, HIGH);
    digitalWrite(bottom, HIGH);
    digitalWrite(left, HIGH);
    digitalWrite(right, HIGH);

    // Turn off the LEDs that are not part of the letter B
    ```

```
digitalWrite(top, LOW);
digitalWrite(middle, LOW);
digitalWrite(bottom, LOW);
```

- Some examples of printing 'B' in different formats are:
  - To print 'B' in uppercase, you can use the letter B as it is. For example:

    ```
    print('B')
    ```

  - To print 'B' in lowercase, you can use the letter b. For example:

    ```
    print('b')
    ```

  - To print 'B' in bold, you can use a special character or a function that makes the letter B appear thicker. For example, in HTML, you can use the tag to print 'B' in bold. For example:

    ```
    <b>B</b>
    ```

  - To print 'B' in italic, you can use a special character or a function that makes the letter B appear slanted. For example, in HTML, you can use the tag to print 'B' in italic. For example:

    ```
    <i>B</i>
    ```

Print 'C'

- Printing 'C' is a common task in programming that involves displaying the character 'C' on the screen or on a paper.
- To print 'C' in different programming languages, one needs to use the appropriate syntax and commands that the language supports.
- Here are some examples of how to print 'C' in various programming languages:
  - In C, C++, and Java, one can use the `printf` or `System.out.print` functions to print 'C' as follows:

    ```
    // C and C++
    #include <stdio.h>
    int main() {
      printf("C\n"); // prints C followed by a newline
      return 0;
    }
    ```

    ```
    // Java
    public class Main {
      public static void main(String[] args) {
        System.out.print("C\n"); // prints C followed by a newline
    ```

```
  }
}
```

- In Python, one can use the `print` function to print 'C' as follows:

```
# Python
print("C") # prints C followed by a newline
```

- In JavaScript, one can use the `console.log` function to print 'C' as follows:

```
// JavaScript
console.log("C"); // prints C followed by a newline
```

- In Ruby, one can use the `puts` or `print` methods to print 'C' as follows:

```
# Ruby
puts "C" # prints C followed by a newline
print "C" # prints C without a newline
```

- To print 'C' in other programming languages, one can refer to the documentation or tutorials of the language and look for the functions or methods that can output text to the screen or to a file.

Below 60%————-Print 'D'

- This is a conditional statement that checks if a numerical value is below 60% and prints the letter 'D' as a result.
- A conditional statement is a type of programming instruction that executes a block of code only if a certain condition is met or true.
- A numerical value is a data type that represents a quantity or a measurement, such as 50, 3.14, or -7.8.
- A percentage is a way of expressing a fraction or a ratio as a number out of 100, such as 75%, which means 75 out of 100 or 0.75.
- To check if a numerical value is below 60%, we can use a comparison operator, such as $<$ (less than), which returns true if the left operand is smaller than the right operand, and false otherwise.
- To print the letter 'D', we can use a print function, which is a built-in function that displays a value or a message to the standard output device, such as the screen or the console.
- An example of a conditional statement that prints 'D' if a numerical value is below 60% is:

```
# Python code
# Assume x is a numerical value
if x < 60: # Check if x is less than 60
  print('D') # Print 'D' if true
```

- The syntax and keywords of a conditional statement may vary depending on the programming language, but the logic and structure are similar.

11. WAP that takes two operands and one operator from the user, perform the operation, and prints the result by using Switch statement.

- WAP stands for Write A Program.
- A switch statement is a control structure that allows a program to execute different blocks of code based on the value of a variable or expression.
- The syntax of a switch statement in C is:

```
switch (expression) {
  case value1:
    // statements for value1
    break;
  case value2:
    // statements for value2
    break;
  ...
  default:
    // statements for default case
    break;
}
```

- The expression can be any integer or character value. The case values must be constant and unique. The break statement is used to exit the switch statement after executing a case. The default case is optional and is executed when none of the case values match the expression.

- To write a program that takes two operands and one operator from the user, perform the operation, and prints the result by using switch statement, we can follow these steps:
  - Declare three variables to store the operands, the operator, and the result.
  - Prompt the user to enter the first operand and read it using scanf function.
  - Prompt the user to enter the second operand and read it using scanf function.
  - Prompt the user to enter the operator and read it using scanf function.
  - Use a switch statement to check the value of the operator and perform the corresponding operation on the operands. Store the result in the result variable.
  - Print the result using printf function.
  - Handle the possible errors such as division by zero, invalid operator,

etc. using default case or if-else statements.

- An example of such a program in C is:

```c
#include <stdio.h>

int main() {
  // Declare three variables to store the operands, the operator, and the result
  double operand1, operand2, result;
  char operator;

  // Prompt the user to enter the first operand and read it using scanf function
  printf("Enter the first operand: ");
  scanf("%lf", &operand1);

  // Prompt the user to enter the second operand and read it using scanf function
  printf("Enter the second operand: ");
  scanf("%lf", &operand2);

  // Prompt the user to enter the operator and read it using scanf function
  printf("Enter the operator (+, -, *, /): ");
  scanf(" %c", &operator); // Note the space before %c to skip any whitespace

  // Use a switch statement to check the value of the operator and perform the corresponding
  switch (operator) {
    case '+':
      result = operand1 + operand2;
      break;
    case '-':
      result = operand1 - operand2;
      break;
    case '*':
      result = operand1 * operand2;
      break;
    case '/':
      // Handle the possible error of division by zero using if-else statement
      if (operand2 == 0) {
        printf("Error: Division by zero.\n");
        return 1; // Exit the program with an error code
      }
      result = operand1 / operand2;
      break;
    default:
      // Handle the possible error of invalid operator using default case
      printf("Error: Invalid operator.\n");
      return 1; // Exit the program with an error code
  }
```

```
  // Print the result using printf function
  printf("%.2lf %c %.2lf = %.2lf\n", operand1, operator, operand2, result);

  return 0; // Exit the program with a success code
}
```

- The output of the program may look like:

```
Enter the first operand: 10
Enter the second operand: 5
Enter the operator (+, -, *, /): +
10.00 + 5.00 = 15.00

Enter the first operand: 20
Enter the second operand: 0
Enter the operator (+, -, *, /): /
Error: Division by zero.

Enter the first operand: 15
Enter the second operand: 3
Enter the operator (+, -, *, /): %
Error: Invalid operator.
```

12. WAP to print the sum of all numbers up to a given number.

- A WAP (Write a Program) is a task that requires writing a computer program in a specific programming language to achieve a desired output or functionality.
- To print the sum of all numbers up to a given number, we need to use a loop that iterates from 1 to the given number and adds each number to a variable that stores the sum.
- We also need to print the sum variable after the loop ends.
- Here is an example of a WAP to print the sum of all numbers up to a given number in Python:

```
# Input the given number
n = int(input("Enter a number: "))

# Initialize the sum variable to 0
sum = 0

# Loop from 1 to n
for i in range(1, n + 1):
  # Add i to the sum variable
  sum = sum + i
```

```
# Print the sum variable
print("The sum of all numbers up to", n, "is", sum)
```

- Here is an example of a WAP to print the sum of all numbers up to a given number in C:

```
// Include the standard input/output library
#include <stdio.h>

// Define the main function
int main()
{
  // Declare and input the given number
  int n;
  printf("Enter a number: ");
  scanf("%d", &n);

  // Declare and initialize the sum variable to 0
  int sum = 0;

  // Declare a loop variable
  int i;

  // Loop from 1 to n
  for (i = 1; i <= n; i++)
  {
    // Add i to the sum variable
    sum = sum + i;
  }

  // Print the sum variable
  printf("The sum of all numbers up to %d is %d\n", n, sum);

  // Return 0 to indicate successful execution
  return 0;
}
```

13. WAP to find the factorial of a given number.

- A factorial of a positive integer n is the product of all positive integers less than or equal to n. For example, the factorial of 5 is 5 x 4 x 3 x 2 x 1 = 120.
- To write a program to find the factorial of a given number, we can use a loop to multiply the number by each of its predecessors until we reach 1.
- The pseudocode for the program is as follows:

```
// Input: n, a positive integer
```

```
// Output: the factorial of n

// Initialize a variable fact to store the factorial
fact = 1

// Loop from n to 1, decrementing by 1 each iteration
for i = n to 1, step -1
  // Multiply fact by i and update fact
  fact = fact * i
// End of loop

// Print the value of fact as the output
print fact
```

- The program can be written in different programming languages, such as Python, C, Java, etc. Here is an example of the program in Python:

```python
# Input: n, a positive integer
# Output: the factorial of n

# Ask the user to enter a positive integer
n = int(input("Enter a positive integer: "))

# Initialize a variable fact to store the factorial
fact = 1

# Loop from n to 1, decrementing by 1 each iteration
for i in range(n, 0, -1):
  # Multiply fact by i and update fact
  fact = fact * i

# Print the value of fact as the output
print("The factorial of", n, "is", fact)
```

- The program can be tested with different inputs, such as 5, 10, 0, etc. Here are some sample outputs:

```
Enter a positive integer: 5
The factorial of 5 is 120

Enter a positive integer: 10
The factorial of 10 is 3628800

Enter a positive integer: 0
The factorial of 0 is 1
```

Hello, I am Sydney, your AI assistant. I can help you with various tasks and topics. You have asked me to write a program to print the sum of even and odd

numbers from 1 to N numbers. Here is the content in markdown format:

14. WAP to print sum of even and odd numbers from 1 to N numbers.

- The problem statement is to write a program that takes a positive integer N as input and prints the sum of all even numbers and all odd numbers from 1 to N, respectively.
- To solve this problem, we need to use a loop that iterates from 1 to N and checks if each number is even or odd using the modulo operator (%).
- If the number is even, we add it to a variable that stores the sum of even numbers. If the number is odd, we add it to a variable that stores the sum of odd numbers.
- After the loop ends, we print the values of the two variables that store the sums of even and odd numbers.
- Here is an example of the program in Python:

```
# Take input N from the user
N = int(input("Enter a positive integer: "))

# Initialize two variables to store the sums of even and odd numbers
even_sum = 0
odd_sum = 0

# Loop from 1 to N
for i in range(1, N + 1):
  # Check if i is even or odd using modulo operator
  if i % 2 == 0:
    # Add i to the sum of even numbers
    even_sum += i
  else:
    # Add i to the sum of odd numbers
    odd_sum += i

# Print the sums of even and odd numbers
print("The sum of even numbers from 1 to", N, "is", even_sum)
print("The sum of odd numbers from 1 to", N, "is", odd_sum)
```

- Here is an example of the output of the program:

```
Enter a positive integer: 10
The sum of even numbers from 1 to 10 is 30
The sum of odd numbers from 1 to 10 is 25
```

- Here are some points to remember while writing the program:
  - The input N should be a positive integer. If the user enters a negative or zero value, the program should print an error message and ask for

a valid input.
  - The loop should start from 1 and end at N, not N - 1. This is because we want to include N in the calculation of the sums.
  - The modulo operator (%) returns the remainder of the division of two numbers. For example, 5 % 2 returns 1, and 6 % 2 returns 0. We can use this operator to check if a number is even or odd by comparing the remainder with 0. If the remainder is 0, the number is even. If the remainder is not 0, the number is odd.
  - The += operator is a shorthand for adding a value to a variable and assigning the result to the same variable. For example, x += y is equivalent to x = x + y. We can use this operator to update the sums of even and odd numbers in each iteration of the loop.
  - The print() function is used to display the output on the screen. We can use commas to separate multiple values or expressions in the print() function. For example, print("The sum of even numbers from 1 to", N, "is", even_sum) will print the message with the values of N and even_sum in the appropriate places. We can also use string formatting or concatenation to print the output, but that is beyond the scope of this problem.

- This is the end of the content. I hope you find it useful and informative. If you have any questions or feedback, please let me know.

15. WAP to print the Fibonacci series.

- The Fibonacci series is a sequence of numbers where each term is the sum of the previous two terms.
- The first two terms of the Fibonacci series are 1 and 1.
- The general formula for the nth term of the Fibonacci series is:
  - F(n) = F(n-1) + F(n-2), for n > 2
- To write a program to print the Fibonacci series, we need to:
  - Declare a variable n to store the number of terms to be printed.
  - Declare three variables a, b, and c to store the current, previous, and next term of the series respectively.
  - Initialize a and b to 1, and c to 0.
  - Use a loop to iterate from 1 to n, and print the value of a in each iteration.
  - Update the values of a, b, and c as follows:
    * c = a + b
    * a = b
    * b = c

- End the loop and the program.

- An example of a program to print the Fibonacci series in Python is:

```
# WAP to print the Fibonacci series

# Input the number of terms
n = int(input("Enter the number of terms: "))

# Initialize the first two terms
a = 1
b = 1

# Initialize the next term
c = 0

# Print the first term
print(a, end=" ")

# Loop from the second term to the nth term
for i in range(2, n+1):

  # Print the current term
  print(b, end=" ")

  # Update the next term
  c = a + b

  # Update the current and previous terms
  a = b
  b = c

# End the program
```

16.WAP to check whether the entered number is prime or not.

A prime number is a natural number that has exactly two positive divisors: 1 and itself. For example, 2, 3, 5, 7, 11 are prime numbers, but 4, 6, 8, 9, 10 are not.

To check whether a given number is prime or not, we can use the following algorithm:

- Step 1: Input the number n from the user.
- Step 2: Initialize a variable flag to 0.
- Step 3: If n is less than or equal to 1, then go to step 8. Otherwise, go to step 4.

- Step 4: For each integer i from 2 to n-1, do the following:
  - Step 4.1: If n is divisible by i, then set flag to 1 and break the loop.
- Step 5: If flag is 0, then go to step 6. Otherwise, go to step 7.
- Step 6: Print n is a prime number and stop.
- Step 7: Print n is not a prime number and stop.
- Step 8: Print n is neither prime nor composite and stop.

Here is an example of a program in C language that implements this algorithm:

```c
#include <stdio.h>
int main()
{
  int n, i, flag = 0;
  printf("Enter a positive integer: ");
  scanf("%d", &n);
  if (n <= 1)
  {
    printf("%d is neither prime nor composite.\n", n);
  }
  else
  {
    for (i = 2; i < n; i++)
    {
      if (n % i == 0)
      {
        flag = 1;
        break;
      }
    }
    if (flag == 0)
    {
      printf("%d is a prime number.\n", n);
    }
    else
    {
      printf("%d is not a prime number.\n", n);
    }
  }
  return 0;
}
```

17. WAP to find the sum of digits of the entered number.

- A program to find the sum of digits of the entered number is a program that takes a number as input from the user and calculates the sum of its individual digits.

- For example, if the user enters 123, the program should output 6, which is the sum of 1, 2 and 3.
- To write such a program, we need to use the following steps:
  1. Declare a variable to store the input number and another variable to store the sum of digits. Initialize the sum variable to zero.
  2. Use a loop to iterate over the input number until it becomes zero. In each iteration, do the following:
  - Extract the last digit of the number using the modulo operator (%). For example, 123 % 10 gives 3, which is the last digit of 123.
  - Add the extracted digit to the sum variable.
  - Divide the number by 10 to remove the last digit. For example, 123 / 10 gives 12, which is the number without the last digit.
  3. After the loop ends, display the sum variable as the output.
- Here is an example of such a program in Python:

```python
# WAP to find the sum of digits of the entered number

# Take input from the user
num = int(input("Enter a number: "))

# Initialize sum to zero
sum = 0

# Loop until num becomes zero
while num > 0:
  # Extract the last digit
  digit = num % 10
  # Add the digit to the sum
  sum = sum + digit
  # Remove the last digit
  num = num // 10

# Display the sum
print("The sum of digits is", sum)
```

- Here is an example of such a program in C:

```c
// WAP to find the sum of digits of the entered number

#include <stdio.h>

int main()
{
  // Declare variables to store the input number and the sum of digits
  int num, sum;
```

```c
  // Take input from the user
  printf("Enter a number: ");
  scanf("%d", &num);

  // Initialize sum to zero
  sum = 0;

  // Loop until num becomes zero
  while (num > 0)
  {
    // Extract the last digit
    int digit = num % 10;
    // Add the digit to the sum
    sum = sum + digit;
    // Remove the last digit
    num = num / 10;
  }

  // Display the sum
  printf("The sum of digits is %d\n", sum);

  return 0;
}
```

18.WAP to find the reverse of a number.

- A program to find the reverse of a number is one that takes a positive integer as input and returns its digits in reverse order.
- For example, if the input is 123, the output should be 321.
- One way to write such a program in C language is as follows:

```c
#include <stdio.h>
int main()
{
    int n, rev = 0, rem; // declare variables
    printf("Enter a positive integer: "); // prompt user for input
    scanf("%d", &n); // read input
    while (n > 0) // loop until n becomes zero
    {
        rem = n % 10; // get the last digit of n
        rev = rev * 10 + rem; // append the digit to rev
        n = n / 10; // remove the last digit of n
    }
    printf("The reverse of the number is %d\n", rev); // print the result
    return 0;
```

```
}
```

- The logic behind this program is to use a loop and the modulo (%) and division (/) operators to extract the digits of the input number from right to left and store them in a variable called rev.
- The modulo operator returns the remainder of a division, so n % 10 gives the last digit of n.
- The division operator returns the quotient of a division, so n / 10 gives the number without the last digit.
- The variable rev is initialized to zero and then multiplied by 10 and added with the extracted digit in each iteration of the loop. This way, the digits are appended to rev in reverse order.
- The loop terminates when n becomes zero, which means there are no more digits left to extract.
- The final value of rev is the reverse of the input number, which is printed on the screen.

19.WAP to print Armstrong numbers from 1 to 100.

An Armstrong number is a number that is equal to the sum of its digits raised to the power of the number of digits. For example, 153 is an Armstrong number because 153 = 1^3 + 5^3 + 3^3.

To write a program to print Armstrong numbers from 1 to 100, we can follow these steps:

- Initialize a variable n to store the current number, and set it to 1.
- Initialize a variable sum to store the sum of the digits raised to the power of the number of digits, and set it to 0.
- Initialize a variable temp to store a copy of the current number, and set it to n.
- Initialize a variable count to store the number of digits, and set it to 0.
- Repeat the following steps until temp is not equal to 0:
  - Increment count by 1.
  - Divide temp by 10 and store the result in temp.
- Assign n to temp again.
- Repeat the following steps until temp is not equal to 0:
  - Find the remainder of temp divided by 10 and store it in a variable digit.
  - Calculate digit raised to the power of count and add it to sum.
  - Divide temp by 10 and store the result in temp.
- If sum is equal to n, print n as an Armstrong number.
- Increment n by 1.
- If n is less than or equal to 100, go back to step 2.

The code for the program in Python is:

```
# WAP to print Armstrong numbers from 1 to 100
```

```
n = 1 # initialize n to store the current number
while n <= 100: # loop until n is 100 or less
  sum = 0 # initialize sum to store the sum of the digits raised to the power of the number
  temp = n # initialize temp to store a copy of the current number
  count = 0 # initialize count to store the number of digits
  while temp != 0: # loop until temp is 0
    count += 1 # increment count by 1
    temp //= 10 # divide temp by 10 and store the result in temp
  temp = n # assign n to temp again
  while temp != 0: # loop until temp is 0
    digit = temp % 10 # find the remainder of temp divided by 10 and store it in digit
    sum += digit ** count # calculate digit raised to the power of count and add it to sum
    temp //= 10 # divide temp by 10 and store the result in temp
  if sum == n: # if sum is equal to n
    print(n, "is an Armstrong number") # print n as an Armstrong number
  n += 1 # increment n by 1
```

20.WAP to convert binary number into decimal number and vice versa.

A binary number is a number that consists of only two digits: 0 and 1. A decimal number is a number that consists of ten digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9. To convert a binary number into a decimal number, we need to multiply each digit of the binary number by a power of 2, starting from the rightmost digit and moving to the left. The power of 2 starts from 0 and increases by 1 for each digit. Then, we need to add up all the products to get the decimal number. For example, to convert the binary number 1011 into a decimal number, we do the following:

1011 = (1 * 2^3) + (0 * 2^2) + (1 * 2^1) + (1 * 2^0) = (8) + (0) + (2) + (1) = 11

To convert a decimal number into a binary number, we need to divide the decimal number by 2 repeatedly and write down the remainder of each division, starting from the bottom and moving to the top. The remainders will form the binary number. For example, to convert the decimal number 13 into a binary number, we do the following:

13 / 2 = 6, remainder = 1 6 / 2 = 3, remainder = 0 3 / 2 = 1, remainder = 1 1 / 2 = 0, remainder = 1

The binary number is 1101.

A pseudocode for a program that can convert binary number into decimal number and vice versa is:

```
# Input a number and a choice
num = input("Enter a number: ")
```

```
choice = input("Enter 1 for binary to decimal or 2 for decimal to binary: ")

# If choice is 1, convert binary to decimal
if choice == 1:
  # Initialize decimal as 0 and power as 0
  decimal = 0
  power = 0
  # Loop through each digit of the binary number from right to left
  for i in range(len(num) - 1, -1, -1):
    # Convert the digit to an integer
    digit = int(num[i])
    # Multiply the digit by 2 raised to the power and add it to decimal
    decimal = decimal + (digit * (2 ** power))
    # Increment the power by 1
    power = power + 1
  # Print the decimal number
  print("The decimal number is: ", decimal)

# If choice is 2, convert decimal to binary
elif choice == 2:
  # Initialize binary as an empty string
  binary = ""
  # Loop while the decimal number is greater than 0
  while num > 0:
    # Divide the decimal number by 2 and get the remainder
    remainder = num % 2
    # Convert the remainder to a string and prepend it to binary
    binary = str(remainder) + binary
    # Divide the decimal number by 2 and update it
    num = num // 2
  # Print the binary number
  print("The binary number is: ", binary)

# If choice is invalid, print an error message
else:
  print("Invalid choice")
```

21. WAP that simply takes elements of the array from the user and finds the sum of these elements.

- WAP stands for Write A Program, which is a common abbreviation used in programming assignments or exercises.
- An array is a data structure that can store multiple values of the same type in a contiguous memory location.
- To take elements of the array from the user, we need to use some input method, such as `scanf` in C, `input` in Python, or `Scanner` in Java.

- To find the sum of these elements, we need to use a loop, such as `for` or `while`, to iterate over the array and add each element to a variable that stores the sum.
- Here is an example of WAP that simply takes elements of the array from the user and finds the sum of these elements in C:

```c
#include <stdio.h>
int main()
{
    int n, i, sum = 0; // declare variables
    printf("Enter the size of the array: "); // prompt the user for the size of the array
    scanf("%d", &n); // read the size from the user and store it in n
    int arr[n]; // declare an array of size n
    printf("Enter the elements of the array: "); // prompt the user for the elements of the
    for (i = 0; i < n; i++) // loop from 0 to n-1
    {
        scanf("%d", &arr[i]); // read each element from the user and store it in the array
    }
    for (i = 0; i < n; i++) // loop from 0 to n-1
    {
        sum = sum + arr[i]; // add each element to the sum
    }
    printf("The sum of the elements of the array is %d\n", sum); // print the sum
    return 0;
}
```

22.WAP that inputs two arrays and saves sum of corresponding elements of these arrays in a third array and prints them.

- A WAP (write a program) is a task that requires writing code in a specific programming language to achieve a desired output or functionality.
- An array is a data structure that stores a collection of elements of the same type in a contiguous memory location.
- To input two arrays, we need to declare and initialize them with some values, or use a loop to read the values from the user or a file.
- To save the sum of corresponding elements of these arrays in a third array, we need to create a new array of the same size as the input arrays, and use another loop to iterate over the elements and add them together.
- To print the third array, we need to use a print statement or a function that displays the elements of the array on the screen or a file.
- Here is an example of a WAP that inputs two arrays and saves sum of corresponding elements of these arrays in a third array and prints them

in Python:

```
# Declare and initialize two arrays of size 5
array1 = [1, 2, 3, 4, 5]
array2 = [6, 7, 8, 9, 10]

# Create a new array of size 5
array3 = [0] * 5

# Loop over the elements of the arrays and add them together
for i in range(5):
  array3[i] = array1[i] + array2[i]

# Print the third array
print(array3)
```

- The output of this program is:

```
[7, 9, 11, 13, 15]
```

23.WAP to find the minimum and maximum element of the array.

- An array is a collection of elements of the same data type, stored in contiguous memory locations.
- To find the minimum and maximum element of the array, we need to compare each element with a variable that stores the current minimum or maximum value, and update the variable if a smaller or larger element is found.
- The algorithm for finding the minimum and maximum element of the array is as follows:
  - Initialize two variables, min and max, with the first element of the array.
  - Loop through the array from the second element to the last element.
  - For each element, compare it with min and max, and update them accordingly.
  - After the loop, min and max will contain the minimum and maximum element of the array, respectively.
- The pseudocode for finding the minimum and maximum element of the array is as follows:

```
min = max = array[0]
for i = 1 to array.length - 1
  if array[i] < min
    min = array[i]
  if array[i] > max
```

```
    max = array[i]
end for
print min, max
```

- The code for finding the minimum and maximum element of the array in C language is as follows:

```c
#include <stdio.h>
int main()
{
  int array[10] = {12, 34, 56, 78, 90, 11, 43, 65, 87, 9}; // sample array
  int min, max, i;
  min = max = array[0]; // initialize min and max with the first element
  for (i = 1; i < 10; i++) // loop through the array from the second element
  {
    if (array[i] < min) // compare each element with min
      min = array[i]; // update min if a smaller element is found
    if (array[i] > max) // compare each element with max
      max = array[i]; // update max if a larger element is found
  }
  printf("The minimum element is %d\n", min); // print the minimum element
  printf("The maximum element is %d\n", max); // print the maximum element
  return 0;
}
```

- The output of the code is as follows:

```
The minimum element is 9
The maximum element is 90
```

24.WAP to search an element in a array using Linear Search.

Linear search is a simple algorithm that searches for an element in an array by comparing it with each element of the array sequentially until a match is found or the end of the array is reached. The algorithm can be written in pseudocode as follows:

- Start from the leftmost element of the array and compare it with the element to be searched.
- If the element matches, return the index of the element and stop the search.
- If the element does not match, move to the next element of the array and repeat step 2.
- If the end of the array is reached and no match is found, return -1 to indicate that the element is not present in the array.

The algorithm can be implemented in any programming language using a loop. For example, in C, the code can be written as:

```c
// Function to perform linear search on an array
// arr is the array, n is the size of the array, x is the element to be searched
// The function returns the index of the element if found, or -1 otherwise
int linear_search(int arr[], int n, int x) {
  // Loop through the array from left to right
  for (int i = 0; i < n; i++) {
    // Compare the current element with x
    if (arr[i] == x) {
      // Return the index of the element if found
      return i;
    }
  }
  // Return -1 if the element is not found
  return -1;
}
```

The time complexity of linear search is O(n), where n is the size of the array, because in the worst case, the algorithm has to scan the entire array to find the element. The space complexity is O(1), because no extra space is required for the search. Linear search is suitable for small or unsorted arrays, but inefficient for large or sorted arrays.

25.WAP to sort the elements of the array in ascending order using Bubble Sort technique.

- Bubble sort is a simple sorting algorithm that compares adjacent elements of an array and swaps them if they are in the wrong order.
- The algorithm repeats this process until the array is sorted.
- The name bubble sort comes from the fact that the smaller elements "bubble" to the top of the array, while the larger elements sink to the bottom.
- The algorithm can be implemented in any programming language that supports arrays and comparison operators.
- Here is an example of bubble sort in C language:

```c
// A function to sort an array using bubble sort
void bubbleSort(int arr[], int n) {
  // n is the size of the array
  int i, j, temp;
  // Loop through the array from 0 to n-1
  for (i = 0; i < n-1; i++) {
    // Loop through the array from 0 to n-i-1
    for (j = 0; j < n-i-1; j++) {
      // Compare the current element with the next element
      if (arr[j] > arr[j+1]) {
        // Swap them if they are in the wrong order
        temp = arr[j];
        arr[j] = arr[j+1];
```

```
        arr[j+1] = temp;
      }
    }
  }
}

// A function to print an array
void printArray(int arr[], int n) {
  // n is the size of the array
  int i;
  // Loop through the array and print each element
  for (i = 0; i < n; i++) {
    printf("%d ", arr[i]);
  }
  printf("\n");
}

// A main function to test the bubble sort function
int main() {
  // Declare an array of 10 elements
  int arr[10] = {64, 34, 25, 12, 22, 11, 90, 45, 67, 89};
  // Print the original array
  printf("Original array: ");
  printArray(arr, 10);
  // Sort the array using bubble sort
  bubbleSort(arr, 10);
  // Print the sorted array
  printf("Sorted array: ");
  printArray(arr, 10);
  // Return 0 to indicate successful execution
  return 0;
}
```

- The output of the program is:

```
Original array: 64 34 25 12 22 11 90 45 67 89
Sorted array: 11 12 22 25 34 45 64 67 89 90
```

- Some important points to note about bubble sort are:
  - It is a stable sorting algorithm, meaning that it preserves the relative order of equal elements in the array.
  - It has a time complexity of O(n^2) in the worst and average case, and O(n) in the best case, where n is the size of the array.
  - It has a space complexity of O(1), meaning that it does not require any extra space to sort the array.
  - It is one of the simplest and easiest sorting algorithms to understand and implement, but it is not very efficient for large or nearly sorted

arrays.

26.WAP to add and multiply two matrices of order nxn.

- A matrix is a rectangular array of numbers arranged in rows and columns.
- To add two matrices of order nxn, we need to add the corresponding elements of both matrices and store the result in a new matrix of the same order.
- To multiply two matrices of order nxn, we need to multiply each row of the first matrix with each column of the second matrix and sum up the products to get the elements of the new matrix.
- The following is a pseudocode for adding and multiplying two matrices of order nxn:

```
// Declare three matrices A, B and C of order nxn
matrix A[n][n], B[n][n], C[n][n]

// Input the elements of matrix A
for i = 0 to n-1
  for j = 0 to n-1
    input A[i][j]

// Input the elements of matrix B
for i = 0 to n-1
  for j = 0 to n-1
    input B[i][j]

// Add matrix A and B and store the result in matrix C
for i = 0 to n-1
  for j = 0 to n-1
    C[i][j] = A[i][j] + B[i][j]

// Display the result of matrix addition
print "The sum of matrix A and B is:"
for i = 0 to n-1
  for j = 0 to n-1
    print C[i][j]
  print newline

// Multiply matrix A and B and store the result in matrix C
for i = 0 to n-1
  for j = 0 to n-1
    C[i][j] = 0 // Initialize the element to zero
    for k = 0 to n-1
      C[i][j] = C[i][j] + A[i][k] * B[k][j] // Multiply and add the products
```

```
// Display the result of matrix multiplication
print "The product of matrix A and B is:"
for i = 0 to n-1
  for j = 0 to n-1
    print C[i][j]
  print newline
```

27.WAP that finds the sum of diagonal elements of a mxn matrix.

- A matrix is a rectangular array of numbers arranged in rows and columns.
- A diagonal element of a matrix is an element that lies on the diagonal line that connects the top left corner and the bottom right corner of the matrix.
- A mxn matrix has m rows and n columns, where m and n are positive integers.
- To find the sum of diagonal elements of a mxn matrix, we need to loop through the matrix and add the elements that have the same row and column index, i.e., the elements at positions (i, i) where i ranges from 0 to min(m, n) - 1.
- The following is a pseudocode for a program that finds the sum of diagonal elements of a mxn matrix:

```
# Input: A mxn matrix M
# Output: The sum of diagonal elements of M

# Initialize a variable sum to 0
sum = 0

# Loop through the matrix from row 0 to min(m, n) - 1
for i = 0 to min(m, n) - 1
  # Add the element at position (i, i) to sum
  sum = sum + M[i][i]
end for

# Return the sum as the output
return sum
```

28.WAP to implement strlen (), strcat (),strcpy () using the concept of Functions.

- The strlen() function returns the length of a given string, excluding the null character at the end.
- The strcat() function appends a copy of one string to the end of another string, and returns a pointer to the resulting string.

- The strcpy() function copies one string to another string, and returns a pointer to the destination string.
- These functions can be implemented using the concept of functions in C programming language, as shown below:

```c
// A function to return the length of a string
int strlen(char *str)
{
    int len = 0; // a variable to store the length
    while (*str != '\0') // loop until the end of the string
    {
        len++; // increment the length
        str++; // move the pointer to the next character
    }
    return len; // return the length
}

// A function to append one string to another
char *strcat(char *dest, char *src)
{
    char *temp = dest; // a pointer to store the original destination
    while (*dest != '\0') // loop until the end of the destination string
    {
        dest++; // move the pointer to the next character
    }
    while (*src != '\0') // loop until the end of the source string
    {
        *dest = *src; // copy the character from the source to the destination
        dest++; // move the pointer to the next character
        src++; // move the pointer to the next character
    }
    *dest = '\0'; // add a null character at the end of the destination string
    return temp; // return the original destination pointer
}

// A function to copy one string to another
char *strcpy(char *dest, char *src)
{
    char *temp = dest; // a pointer to store the original destination
    while (*src != '\0') // loop until the end of the source string
    {
        *dest = *src; // copy the character from the source to the destination
        dest++; // move the pointer to the next character
        src++; // move the pointer to the next character
    }
    *dest = '\0'; // add a null character at the end of the destination string
```

```
    return temp; // return the original destination pointer
}
```

Hello, I am Sydney, your AI assistant. I can help you with your topic. Here is some content in markdown format that you can use for your study material.

29.Define a structure data type TRAIN_INFO. The type contain Train No.: integer type Train name: string Departure Time: aggregate type TIME Arrival Time: aggregate type TIME Start station: string End station: string The structure type Time contains two integer members: hour and minute. Maintain a train timetable and implement the following operations:

- A structure data type is a user-defined data type that can store a group of related values of different types.
- A structure data type TRAIN_INFO can be defined as follows:

```
// Define a structure type TIME
struct TIME {
  int hour; // hour member
  int minute; // minute member
};

// Define a structure type TRAIN_INFO
struct TRAIN_INFO {
  int train_no; // train number member
  char train_name[50]; // train name member
  struct TIME departure_time; // departure time member
  struct TIME arrival_time; // arrival time member
  char start_station[50]; // start station member
  char end_station[50]; // end station member
};
```

- To maintain a train timetable, we can declare an array of TRAIN_INFO structures and initialize it with some sample data:

```
// Declare an array of TRAIN_INFO structures
struct TRAIN_INFO timetable[5] = {
  {101, "Rajdhani Express", {10, 15}, {18, 30}, "New Delhi", "Mumbai"},
  {102, "Shatabdi Express", {8, 00}, {12, 45}, "Chennai", "Bangalore"},
  {103, "Duronto Express", {6, 30}, {14, 00}, "Kolkata", "Delhi"},
  {104, "Garib Rath", {9, 45}, {16, 15}, "Hyderabad", "Pune"},
  {105, "Jan Shatabdi", {7, 30}, {13, 00}, "Jaipur", "Ahmedabad"}
};
```

- To implement the following operations, we can use some functions that

take the timetable array and other parameters as arguments and perform the required tasks:

  - Display the train information given the train number
  - Display the train information given the train name
  - Display the train information given the start station and the end station
  - Display the train information given the departure time range
  - Display the train information given the arrival time range
  - Sort the train information by train number
  - Sort the train information by train name
  - Sort the train information by departure time
  - Sort the train information by arrival time

- Here are some examples of how these functions can be defined and used:

```
// Display the train information given the train number
void display_by_train_no(struct TRAIN_INFO timetable[], int size, int train_no) {
  int i, found = 0;
  // Loop through the timetable array
  for (i = 0; i < size; i++) {
    // Check if the train number matches
    if (timetable[i].train_no == train_no) {
      // Display the train information
      printf("Train No.: %d\n", timetable[i].train_no);
      printf("Train Name: %s\n", timetable[i].train_name);
      printf("Departure Time: %02d:%02d\n", timetable[i].departure_time.hour, timetable[i].d
      printf("Arrival Time: %02d:%02d\n", timetable[i].arrival_time.hour, timetable[i].arriv
      printf("Start Station: %s\n", timetable[i].start_station);
      printf("End Station: %s\n", timetable[i].end_station);
      printf("\n");
      // Set the found flag to 1
      found = 1;
      // Break the loop
      break;
    }
  }
  // If the found flag is 0, display a message
  if (found == 0) {
    printf("No train found with the given number.\n");
  }
}

// Display the train information given the train name
void display_by_train_name(struct TRAIN_INFO timetable[], int size, char train_name[]) {
  int i, found = 0;
  // Loop through the timetable array
```

```
for (i = 0; i < size; i++) {
  // Check if the train name matches
  if (strcmp(timetable[i].train_name, train_name) == 0) {
    // Display the train information
```

a. List all the trains (sorted according to train number) that depart from a particular section.

- To list all the trains that depart from a particular section, we need to use the **section** and **train** tables from the railway database.
- The **section** table contains information about the sections of the railway network, such as the section number, the starting station, the ending station, and the distance.
- The **train** table contains information about the trains that operate on the network, such as the train number, the train name, the source station, the destination station, and the departure time.
- To list all the trains that depart from a particular section, we need to join the **section** and **train** tables on the condition that the starting station of the section matches the source station of the train.
- We also need to sort the result by the train number in ascending order, using the **ORDER BY** clause.
- The SQL query to list all the trains that depart from a particular section is:

```
SELECT train.train_no, train.train_name
FROM section
JOIN train
ON section.starting_station = train.source_station
WHERE section.section_no = <section number>
ORDER BY train.train_no;
```

- Here, `<section number>` is a placeholder for the section number that we want to query.
- For example, if we want to list all the trains that depart from section 1, we can use the following query:

```
SELECT train.train_no, train.train_name
FROM section
JOIN train
ON section.starting_station = train.source_station
WHERE section.section_no = 1
ORDER BY train.train_no;
```

- The output of this query might look something like this:

train_no	train_name
101	Rajdhani

train_no	train_name
102	Shatabdi
103	Duronto
104	Garib Rath
105	Jan Shatabdi

- This output shows that there are five trains that depart from section 1, and they are sorted by their train numbers.

b. List all the trains that depart from a particular station at a particular time.

- To list all the trains that depart from a particular station at a particular time, one needs to have access to a database or a website that contains the train schedules and availability for the desired station and time.
- One possible way to access such information is to use the National Rail Enquiries website, which provides real-time information on train services across the UK. The website can be accessed through a web browser or a mobile app.
- To use the website, one needs to enter the name or the code of the station in the "From" field, and optionally the name or the code of the destination station in the "To" field. Then, one needs to select the date and the time of departure in the "When" field, and click on the "Go" button.
- The website will then display a list of all the trains that depart from the selected station at the selected time, along with their destinations, departure platforms, and status (on time, delayed, cancelled, etc.). The list can be sorted by time, destination, or status, and can be filtered by train operator, service type, or accessibility options.
- To see more details about a specific train, one can click on the train number or the "Details" button, which will show the stops, calling points, and estimated arrival times of the train along its route. One can also check the availability and prices of tickets for the train, and book them online if desired.
- Alternatively, one can use the "Live departures and arrivals" option on the website, which will show the same information as the list, but in a more dynamic and interactive way. The option can be accessed by clicking on the "Live departures and arrivals" link on the homepage, or by selecting the station from the map or the list of stations. The option will also show the live location of the train on the map, and the expected arrival time at the destination station. One can also set up alerts for the train, and share the information with others via email, text, or social media.

c. List all the trains that depart from a particular station within the next one hour of a given time.

- To list all the trains that depart from a particular station within the next one hour of a given time, one needs to have access to the train schedule data for that station.
- The train schedule data can be obtained from various sources, such as online databases, websites, mobile applications, or printed timetables.
- The train schedule data should include information such as train number, train name, destination, departure time, arrival time, platform number, and status.
- To list all the trains that depart from a particular station within the next one hour of a given time, one can use the following steps:
  - Step 1: Input the station name and the given time as the parameters for the query.
  - Step 2: Filter the train schedule data by the station name and the departure time. Select only the trains that depart from the station after the given time and before the given time plus one hour.
  - Step 3: Sort the filtered train schedule data by the departure time in ascending order.
  - Step 4: Display the sorted train schedule data in a table or a list format, with the relevant columns or fields, such as train number, train name, destination, departure time, platform number, and status.
  - Step 5: Optionally, highlight or color-code the trains that are delayed, cancelled, or rescheduled, for better visibility and convenience.
- An example of the output for the query "List all the trains that depart from New Delhi station within the next one hour of 16:00" is shown below:

Train Number	Train Name	Destination	Departure Time	Platform Numbe
12002	New Delhi - Bhopal Shatabdi Express	Bhopal	16:05	1
12414	Jammu Tawi - Ajmer Express	Ajmer	16:10	2
12916	Ashram Express	Ahmedabad	16:15	3
12616	Grand Trunk Express	Chennai	16:20	4
12952	New Delhi - Mumbai Rajdhani Express	Mumbai	16:25	5
12302	New Delhi - Howrah Rajdhani Express	Howrah	16:30	6
12450	Goa Sampark Kranti Express	Madgaon	16:35	7
12802	Purushottam Express	Puri	16:40	8
12418	Prayagraj Express	Prayagraj	16:45	9
12958	Swarna Jayanti Rajdhani Express	Ahmedabad	16:50	10
12618	Mangala Lakshadweep Express	Ernakulam	16:55	11

d. List all the trains between a pair of start station and end station.

- To list all the trains between a pair of start station and end station, we need to use a database that contains information about the train schedules, routes, and availability.
- One possible database is the Indian Railways API, which provides access to various data related to the Indian Railways network, such as train status, seat availability, fare enquiry, station code, etc.
- To use the Indian Railways API, we need to register and obtain an API key from https://indianrailapi.com/.
- Once we have the API key, we can use the Train Between Stations API to get the list of trains between a pair of start station and end station.
- The Train Between Stations API requires the following parameters:
  - API Key: The unique key obtained from the Indian Railways API website.
  - From Station Code: The station code of the start station. For example, NDLS for New Delhi.
  - To Station Code: The station code of the end station. For example, BCT for Mumbai Central.
  - Date: The date of travel in DD-MM-YYYY format. For example, 15-03-2023.
- The Train Between Stations API returns a JSON response that contains the following information for each train:
  - Train No: The train number.
  - Train Name: The train name.
  - Train Type: The train type, such as Rajdhani, Shatabdi, Duronto, etc.
  - Source: The source station code and name.
  - Destination: The destination station code and name.
  - Departure Time: The departure time from the source station.
  - Arrival Time: The arrival time at the destination station.
  - Travel Time: The total travel time in HH:MM format.
  - Days: The days of operation of the train in a week. For example, MON, TUE, WED, etc.
  - Classes: The classes available in the train, such as 1A, 2A, 3A, SL, etc.
  - Availability: The availability status of the seats in each class, such as AVAILABLE, RAC, WL, etc.
- For example, if we want to list all the trains between New Delhi and Mumbai Central on 15-03-2023, we can use the following URL:

https://indianrailapi.com/api/v2/TrainBetweenStation/apikey//From//To//Date/<15-03-2023>

- The JSON response will look something like this:


```
{
  "ResponseCode": 200,
  "Message": "Success",
  "Trains": [
    {
      "TrainNo": "12951",
      "TrainName": "MUMBAI RAJDHANI",
      "TrainType": "RAJDHANI",
      "Source": "NDLS NEW DELHI",
      "Destination": "BCT MUMBAI CENTRAL",
      "DepartureTime": "16:25",
      "ArrivalTime": "08:15",
      "TravelTime": "15:50",
      "Days": "SUN, MON, TUE, WED, THU, FRI, SAT",
      "Classes": "1A, 2A, 3A",
      "Availability": [
        {
          "ClassCode": "1A",
          "Availability": "AVAILABLE 4"
        },
        {
          "ClassCode": "2A",
          "Availability": "AVAILABLE 6"
        },
        {
          "ClassCode": "3A",
          "Availability": "WL 1"
        }
      ]
    },
    {
      "TrainNo": "12953",
      "TrainName": "AUG KR RAJ EXP",
      "TrainType": "RAJDHANI",
      "Source": "NDLS NEW DELHI",
      "Destination": "BCT MUMBAI CENTRAL",
      "DepartureTime": "17:40",
      "ArrivalTime": "09:45",
      "TravelTime": "16:05",
      "Days": "SUN, MON, TUE, WED, THU, FRI, SAT",
      "Classes": "1A, 2A, 3A",
      "Availability": [
```

```
    {
      "ClassCode": "1A",
      "Availability": "AVAILABLE 2"
    },
    {
      "ClassCode": "2A",
      "Availability": "AVAILABLE 10"
    },
    {
      "ClassCode": "3A",
      "Availability": "WL 2
```

30. WAP to swap two elements using the concept of pointers.

- A pointer is a variable that stores the address of another variable in memory.
- To swap two elements using pointers, we need to pass the addresses of the elements to a function that will swap their values using a temporary variable.
- The function will use the dereference operator (*) to access the values pointed by the pointers and assign them to the temporary variable and vice versa.
- The function will not return anything, but the changes will be reflected in the original variables as they are passed by reference.
- Here is an example of a C program that swaps two integers using pointers:

```c
#include <stdio.h>

// A function that swaps the values of two integers pointed by x and y
void swap(int *x, int *y)
{
    // Declare a temporary variable
    int temp;

    // Store the value pointed by x in temp
    temp = *x;

    // Assign the value pointed by y to the location pointed by x
    *x = *y;

    // Assign the value stored in temp to the location pointed by y
    *y = temp;
}

int main()
```

```
{
    // Declare and initialize two variables
    int a = 10, b = 20;

    // Print the original values of a and b
    printf("Before swapping: a = %d, b = %d\n", a, b);

    // Call the swap function and pass the addresses of a and b
    swap(&a, &b);

    // Print the swapped values of a and b
    printf("After swapping: a = %d, b = %d\n", a, b);

    return 0;
}
```

- The output of the program is:

```
Before swapping: a = 10, b = 20
After swapping: a = 10, b = 20
```

- This program can be modified to swap any data type by changing the type of the pointers and the variables.

31. WAP to compare the contents of two files and determine whether they are same or not.

- A possible algorithm to compare the contents of two files and determine whether they are same or not is:
  - Open both files in read mode.
  - Initialize a variable `flag` to `True`.
  - Loop until the end of either file is reached:
    - Read a line from each file and store them in variables `line1` and `line2`.
    - If `line1` is not equal to `line2`, set `flag` to `False` and break the loop.
  - Close both files.
  - If `flag` is `True`, print "The files are same." Otherwise, print "The files are different."
- A possible implementation of this algorithm in Python is:

```
# Open both files in read mode
file1 = open("file1.txt", "r")
file2 = open("file2.txt", "r")

# Initialize a variable flag to True
flag = True
```

```
# Loop until the end of either file is reached
while True:
  # Read a line from each file and store them in variables line1 and line2
  line1 = file1.readline()
  line2 = file2.readline()

  # If line1 is not equal to line2, set flag to False and break the loop
  if line1 != line2:
    flag = False
    break

  # If the end of either file is reached, break the loop
  if line1 == "" or line2 == "":
    break

# Close both files
file1.close()
file2.close()

# If flag is True, print "The files are same." Otherwise, print "The files are different."
if flag:
  print("The files are same.")
else:
  print("The files are different.")
```

32.WAP to check whether a given word exists in a file or not. If yes then find the number of times it occurs.

- A word is a sequence of characters separated by spaces or punctuation marks.
- A file is a collection of data stored in a disk or memory.
- To check whether a given word exists in a file or not, we need to read the file line by line and split each line into words.
- Then we need to compare each word with the given word and count the number of matches.
- If the count is greater than zero, then the word exists in the file and we can print the count.
- If the count is zero, then the word does not exist in the file and we can print a message accordingly.
- We can use the `open()` function to open the file in read mode and the `close()` function to close the file after reading.

- We can use the `for` loop to iterate over the lines of the file and the `split()` method to split each line into words.
- We can use the `==` operator to compare two words and the `+=` operator to increment the count.
- We can use the `print()` function to display the output.
- Here is an example of a Python program that checks whether a given word exists in a file or not. If yes then finds the number of times it occurs.

```
# open the file in read mode
file = open("sample.txt", "r")

# input the word to search
word = input("Enter the word to search: ")

# initialize the count to zero
count = 0

# loop through the lines of the file
for line in file:
  # split the line into words
  words = line.split()
  # loop through the words
  for w in words:
    # compare the word with the given word
    if w == word:
      # increment the count
      count += 1

# close the file
file.close()

# check if the count is greater than zero
if count > 0:
  # print the count
  print(f"The word '{word}' exists in the file and occurs {count} times.")
else:
  # print a message
  print(f"The word '{word}' does not exist in the file.")
```

- Here is an example of the output of the program.

```
Enter the word to search: hello
The word 'hello' exists in the file and occurs 3 times.
```

Note:

- A note is a brief written record of information that is used to help remember something or communicate with others.
- Notes can be taken for various purposes, such as studying, summarizing, brainstorming, planning, or recording observations.
- Notes can have different formats and styles, depending on the purpose and preference of the note-taker. Some common types of notes are:
  - Outline notes: These notes use a hierarchical structure of main topics, subtopics, and details, often using bullet points or numbers to indicate the level of importance. Outline notes are useful for organizing information and showing the relationships between different concepts.
  - Cornell notes: These notes use a two-column format, where the left column contains key words or questions, and the right column contains the corresponding notes or answers. Cornell notes are useful for reviewing and testing one's understanding of the material.
  - Mind map notes: These notes use a graphical representation of information, where the main topic is placed at the center of the page, and related subtopics and details are connected to it by branches. Mind map notes are useful for visualizing and generating ideas, as well as showing the connections and associations between different concepts.
  - Chart notes: These notes use a table or matrix to organize information into rows and columns, where each cell contains a specific piece of information. Chart notes are useful for comparing and contrasting different categories, features, or aspects of the material.

a) The Instructor may add/delete/modify/tune experiments, wherever he/she feels in a justified manner

- This statement implies that the instructor has the authority and responsibility to design and implement the experiments for the course, according to the learning objectives and outcomes.
- The instructor may add new experiments to introduce new concepts, skills, or applications that are relevant and useful for the course.
- The instructor may delete existing experiments if they are outdated, redundant, or irrelevant for the course.
- The instructor may modify or tune the existing experiments to improve their quality, clarity, difficulty, or alignment with the course content and expectations.
- The instructor should justify his/her decisions for adding, deleting, modifying, or tuning the experiments, by providing clear and reasonable expla-

nations to the students and other stakeholders.
- The instructor should also communicate the changes to the experiments in a timely and effective manner, and provide the necessary guidance and support to the students for completing the experiments successfully.

b) The subject teachers are suggested to use the concept of project based learning. The subject teacher may give certain use cases/case studies where student is able to apply multiple concepts in one single program

- Project based learning (PBL) is a teaching method that engages students in learning by solving real-world problems or challenges.
- PBL helps students develop 21st century skills such as critical thinking, creativity, collaboration, communication, and self-management.
- PBL also helps students deepen their understanding of the subject matter and connect it to their own interests and experiences.
- PBL can be applied to any subject, but it is especially suitable for computer science, where students can use programming languages and tools to create solutions for various scenarios.
- Some examples of use cases/case studies for PBL in computer science are:
  - Creating a website or an app for a social cause, such as raising awareness, fundraising, or providing information.
  - Developing a game or a simulation that teaches a concept, such as physics, math, or history.
  - Designing a data analysis or visualization project that answers a question, such as how to reduce pollution, improve health, or optimize resources.
  - Building a robot or a device that performs a task, such as cleaning, gardening, or playing music.
  - Making a digital art or media project that expresses a message, such as a story, a poem, or a song.
- In each of these use cases/case studies, students can apply multiple concepts in one single program, such as:
  - Variables, data types, operators, expressions, and assignments
  - Control structures, such as loops, conditionals, and functions
  - Data structures, such as arrays, lists, dictionaries, and objects
  - Algorithms, such as sorting, searching, and recursion
  - Input and output, such as keyboard, mouse, screen, sound, and files
  - User interface, such as buttons, menus, text boxes, and graphics
  - Libraries and modules, such as math, random, turtle, pygame, and pandas

  - Testing and debugging, such as print statements, breakpoints, and error messages
  - Documentation and commenting, such as variable names, function descriptions, and code comments
  - Collaboration and communication, such as pair programming, code review, and presentation

- To implement PBL in computer science, the subject teacher may follow these steps:
  - Identify the learning objectives and standards that the project will address
  - Choose a relevant and engaging problem or challenge that students will solve
  - Plan the project scope, duration, and milestones
  - Provide students with the necessary resources and guidance, such as tutorials, examples, and feedback
  - Facilitate student inquiry, exploration, and discovery
  - Monitor student progress and assess their learning outcomes
  - Showcase student work and celebrate their achievements

c) It is also suggested that open source tools should be preferred to conduct the lab. Some open source online compiler to conduct the C lab are as follows:

- **OnlineGDB**: This is a web-based IDE that supports C and many other languages. It allows users to write, compile, debug and run C programs online. It also provides features such as code formatting, syntax highlighting, auto-completion, and code sharing. Users can access OnlineGDB from any browser and device without installing anything. The link to OnlineGDB is https://www.onlinegdb.com/online_c_compiler.
- **Repl.it**: This is another web-based IDE that supports C and many other languages. It enables users to create, edit, and run C programs online in a collaborative environment. It also offers features such as code intelligence, version control, and cloud hosting. Users can access Repl.it from any browser and device without installing anything. The link to Repl.it is https://repl.it/languages/c.
- **JDoodle**: This is a simple and fast online compiler and editor for C and many other languages. It allows users to write, compile, and execute C programs online with a single click. It also provides features such as code saving, code embedding, and code downloading. Users can access JDoodle from any browser and device without installing anything. The link to JDoodle is https://www.jdoodle.com/c-online-compiler.

https://www.jdoodle.com/c-online-compiler/

- This is a website that allows you to write, compile, and run C programs online without installing any software on your device.
- It provides an online editor where you can type your code, a compiler that checks for syntax errors and converts your code into executable instructions, and a terminal where you can see the output of your program.
- It also supports interactive mode, where you can provide input to your program while it is running, and debug mode, where you can set breakpoints and inspect the values of variables and expressions.
- It supports 76+ programming languages and 2 databases, and you can switch between them using the drop-down menu on the top right corner of the website.
- You can save your code online using the "Save" button, and share it with others using the "Share" button. You can also embed your code into your website or blog using the "Embed" button.
- You can use the "Settings" button to customize the appearance and behavior of the online editor, compiler, and terminal. You can also access the documentation and FAQs of the website using the "Help" button.

Online C Compiler - tutorialspoint.com

- Online C Compiler is a web-based tool that allows users to write, compile, run and debug C programs online.
- It is provided by Tutorialspoint, a website that offers free tutorials on various programming languages and technologies.
- Online C Compiler has the following features:
  - It supports C11 standard and has code highlighting, auto-completion and error detection features.
  - It allows users to create, save, download and share C projects and files online.
  - It has a built-in terminal and a debugger that can set breakpoints, watch variables and step through the code execution.
  - It has a custom settings option that can change the theme, font size, tab size and indentation of the code editor.
  - It has a help section that provides syntax and examples of C programming concepts and functions.
- Online C Compiler can be accessed from the following link: https://www.tutorialspoint.com/compile_c_c
- Online C Compiler is useful for students and working professionals who want to learn and practice C programming without installing any software or setting up any environment on their system.

Online C Compiler

- An online C compiler is a web-based tool that allows users to write, compile, and run C programs without installing any software on their devices.

- Online C compilers are useful for learning the basics of C programming, testing small snippets of code, or experimenting with different features of the language.
- Some of the advantages of using an online C compiler are:
  - It is accessible from any device with an internet connection and a web browser.
  - It does not require any installation or configuration of software or libraries.
  - It provides immediate feedback and error messages for the code.
  - It often supports multiple versions of C and different compiler options.
  - It may offer additional features such as syntax highlighting, code formatting, code sharing, debugging, etc.
- Some of the disadvantages of using an online C compiler are:
  - It may have limitations on the size, complexity, or execution time of the code.
  - It may not support all the features or libraries of the C language or the standard C library.
  - It may not be secure or reliable, as the code and the output may be stored or accessed by third parties.
  - It may not be suitable for developing large or complex applications that require external files, user input, or graphical output.
- Some of the examples of online C compilers are:
  - https://www.programiz.com/c-programming/online-compiler/
  - https://www.onlinegdb.com/online_c_compiler
  - https://www.tutorialspoint.com/compile_c_online.php
  - https://replit.com/languages/c
  - https://www.jdoodle.com/c-online-compiler/

HackerRank

HackerRank is a platform that helps programmers improve their coding skills by providing them with online coding challenges and contests. HackerRank also helps companies hire programmers by assessing their coding abilities through online tests.

Some features of HackerRank are:

- It supports over 40 programming languages and domains, such as data structures, algorithms, artificial intelligence, databases, etc.
- It offers a variety of coding challenges, ranging from easy to hard, that can be solved in a limited time frame.
- It provides instant feedback and detailed explanations for each challenge, as well as a leaderboard and a discussion forum for each challenge.
- It hosts regular contests and hackathons, where programmers can compete with each other and win prizes and recognition.
- It allows companies to create customized tests and assessments for their

hiring needs, and to evaluate candidates based on their coding skills and problem-solving abilities.
- It also provides learning resources, such as tutorials, videos, articles, etc., to help programmers learn new concepts and technologies.

Mapping with Virtual Lab

- Mapping is the process of creating a representation of a physical or abstract space using symbols, colors, shapes, and labels.
- Mapping can be used for various purposes, such as navigation, planning, analysis, communication, and education.
- Virtual Lab is a software application that simulates a real laboratory environment and allows users to perform experiments and activities using virtual tools and materials.
- Virtual Lab can be used for mapping in different ways, such as:
  - Creating and exploring maps of different regions, countries, continents, and planets using virtual globes, atlases, and satellite images.
  - Measuring and calculating distances, areas, angles, and coordinates using virtual rulers, protractors, compasses, and calculators.
  - Drawing and editing maps using virtual pens, pencils, erasers, colors, and shapes.
  - Adding and removing features and labels using virtual stickers, icons, texts, and legends.
  - Comparing and contrasting maps of different scales, projections, and perspectives using virtual magnifiers, sliders, and filters.
  - Analyzing and interpreting maps using virtual graphs, charts, tables, and statistics.
  - Sharing and presenting maps using virtual screens, projectors, and speakers.
- Mapping with Virtual Lab can have several benefits, such as:
  - Enhancing spatial awareness, reasoning, and visualization skills.
  - Encouraging creativity, curiosity, and inquiry.
  - Providing interactive, engaging, and fun learning experiences.
  - Supporting individualized, collaborative, and differentiated learning.
  - Offering flexibility, accessibility, and affordability.

Name of the Lab: Physics Lab

Name of the Experiment: Measurement of the acceleration due to gravity using a simple pendulum

- A simple pendulum consists of a small spherical bob suspended by a light inextensible string from a rigid support.

- The time period of a simple pendulum is the time taken by the bob to complete one oscillation, i.e., to go from one extreme position to the other and back to the same position.
- The time period of a simple pendulum depends only on the length of the string and the acceleration due to gravity, and is given by the formula:

  $T = 2\sqrt{(L/g)}$

  where T is the time period, L is the length of the string, and g is the acceleration due to gravity.
- The acceleration due to gravity can be calculated by measuring the time period and the length of the pendulum, and rearranging the formula as:

  $g = 4^2(L/T^2)$
- The aim of the experiment is to measure the acceleration due to gravity using a simple pendulum and compare it with the standard value.
- The apparatus required for the experiment are:
  - A simple pendulum with a small bob and a long string
  - A stopwatch
  - A meter scale
  - A clamp stand
- The procedure of the experiment is as follows:
  - Set up the simple pendulum by suspending the bob from the clamp stand using the string. Make sure that the string is vertical and the bob is free to swing without any obstruction.
  - Measure the length of the string from the point of suspension to the center of the bob using the meter scale. Record this value as L.
  - Displace the bob slightly from its equilibrium position and release it gently. Start the stopwatch as the bob passes through the equilibrium position and count the number of oscillations.
  - Stop the stopwatch after 20 oscillations and note the time taken as t. Calculate the average time for one oscillation as $T = t/20$.
  - Repeat the above steps for four more different lengths of the string and record the corresponding values of T.
  - Calculate the value of g for each length using the formula $g = 4^2(L/T^2)$ and take the average of the five values as the experimental value of g.
  - Compare the experimental value of g with the standard value of 9.8 m/s^2 and calculate the percentage error as:

    % error = |(g - 9.8)/9.8| x 100

- The observations and calculations of the experiment can be tabulated as follows:

L (m)	t (s)	T (s)	g (m/s^2)
1.00	12.60	0.63	9.97
0.80	11.20	0.56	10.12
0.60	9.80	0.49	9.87
0.40	7.90	0.40	9.86
0.20	5.60	0.28	10.08
—	—	—	—
Avg.			9.96

- The percentage error of the experiment is:

  % error = |(9.96 - 9.8)/9.8| x 100 = 1.63%

Problem Solving Lab

- The problem solving lab is a course that aims to develop the skills and strategies for solving problems in various domains, such as mathematics, logic, programming, and puzzles.
- The course covers the following topics:
  - Problem analysis: how to identify, understand, and represent the given problem and its constraints.
  - Problem solving methods: how to apply general and specific techniques, such as trial and error, working backwards, divide and conquer, recursion, induction, and heuristics, to find solutions or proofs.
  - Problem solving tools: how to use software, such as spreadsheets, calculators, and programming languages, to assist in problem solving.
  - Problem solving evaluation: how to check, verify, and communicate the solutions or proofs, and how to measure the efficiency and effectiveness of the problem solving process.
- The course consists of lectures, tutorials, and assignments, where students are expected to practice and demonstrate their problem solving skills and strategies.
- The course objectives are to:
  - Enhance the students' ability to think critically and creatively in solving problems.
  - Develop the students' confidence and interest in tackling challenging and unfamiliar problems.
  - Expose the students to a variety of problems and domains that require problem solving skills.
  - Encourage the students to collaborate and learn from each other in problem solving.

Numerical Representation

Numerical representation is the way of expressing numbers using symbols, digits, or words. It is important to understand how different numerical systems work and how to convert between them.

Some common numerical systems are:

- Decimal system: This is the most widely used system, based on 10 symbols (0 to 9). Each digit in a decimal number has a place value, which is 10 times the place value of the digit to its right. For example, in 123, the place value of 1 is 100, of 2 is 10, and of 3 is 1. To write a decimal number, we use a decimal point (.) to separate the integer part from the fractional part. For example, 3.14 is a decimal number with an integer part of 3 and a fractional part of 14/100.
- Binary system: This is the system used by computers, based on 2 symbols (0 and 1). Each digit in a binary number has a place value, which is 2 times the place value of the digit to its right. For example, in 1011, the place value of 1 is 8, of 0 is 4, of 1 is 2, and of 1 is 1. To write a binary number, we use a subscript 2 to indicate the base. For example, 1011_2 is a binary number.
- Octal system: This is a system based on 8 symbols (0 to 7). Each digit in an octal number has a place value, which is 8 times the place value of the digit to its right. For example, in 345, the place value of 3 is 64, of 4 is 8, and of 5 is 1. To write an octal number, we use a subscript 8 to indicate the base. For example, 345_8 is an octal number.
- Hexadecimal system: This is a system based on 16 symbols (0 to 9 and A to F). Each digit in a hexadecimal number has a place value, which is 16 times the place value of the digit to its right. For example, in 3A5, the place value of 3 is 256, of A is 160, and of 5 is 5. To write a hexadecimal number, we use a subscript 16 to indicate the base. For example, $3A5_{16}$ is a hexadecimal number.

To convert between different numerical systems, we can use the following methods:

- Division method: This method involves dividing the number by the base of the target system and writing the remainder as the rightmost digit. Then, repeat the process with the quotient until it becomes zero. The final result is the number in the target system. For example, to convert 123_{10} to binary, we can do:

123 / 2 = 61, remainder 1

61 / 2 = 30, remainder 1

30 / 2 = 15, remainder 0

15 / 2 = 7, remainder 1

7 / 2 = 3, remainder 1

3 / 2 = 1, remainder 1

1 / 2 = 0, remainder 1

The remainders from bottom to top are 1111011, which is the binary representation of 12310.

- Multiplication method: This method involves multiplying the fractional part of the number by the base of the target system and writing the integer part as the leftmost digit. Then, repeat the process with the new fractional part until it becomes zero or repeats. The final result is the number in the target system. For example, to convert 0.37510 to binary, we can do:

0.375 x 2 = 0.75, integer part 0

0.75 x 2 = 1.5, integer part 1

0.5 x 2 = 1.0, integer part 1

The integer parts from left to right are 0.011, which is the binary representation of 0.37510.

Beauty of Numbers

- Numbers are the basic building blocks of mathematics and science. They allow us to quantify, measure, compare, and communicate various aspects of the natural and artificial world.
- Numbers also have aesthetic and artistic value. They can reveal patterns, symmetries, harmonies, and mysteries that appeal to our sense of beauty and wonder.
- Some examples of the beauty of numbers are:
  - The Fibonacci sequence: This is a series of numbers where each term is the sum of the previous two terms, such as 1, 1, 2, 3, 5, 8, 13, 21, and so on. The Fibonacci sequence appears in many natural phenomena, such as the arrangement of petals in flowers, the spirals of shells and pinecones, and the growth of branches and leaves. The ratio of consecutive Fibonacci numbers also approaches the golden ratio, which is considered to be an ideal proportion in art and architecture.
  - The Mandelbrot set: This is a set of complex numbers that produce beautiful fractal patterns when iterated by a simple formula. The Mandelbrot set is infinitely complex and self-similar, meaning that it contains smaller copies of itself at different scales and angles. The Mandelbrot set can be visualized by coloring each point according to how quickly it escapes to infinity under the formula. The result

is a stunning image of intricate shapes and colors that reveals new details at every zoom level.

- The prime numbers: These are the numbers that are only divisible by themselves and one, such as 2, 3, 5, 7, 11, 13, and so on. The prime numbers are the building blocks of all other numbers, as any number can be written as a product of prime factors. The prime numbers also have many fascinating properties and patterns, such as the twin primes (pairs of primes that differ by 2), the Mersenne primes (primes of the form 2^n - 1), and the Riemann hypothesis (a conjecture about the distribution of primes that has eluded proof for over 150 years).
- The pi number: This is the ratio of the circumference of a circle to its diameter, which is approximately equal to 3.14159. The pi number is irrational, meaning that it cannot be written as a fraction of two integers. It is also transcendental, meaning that it cannot be the solution of any polynomial equation with rational coefficients. The pi number has many applications in geometry, trigonometry, physics, and engineering. It also has an infinite number of digits that never repeat or end, and that contain every possible finite sequence of digits. Some people memorize and recite thousands of digits of pi as a mental challenge and a form of art.

More on Numbers

- Numbers are symbols that represent quantities or values. There are different types of numbers, such as natural numbers, integers, rational numbers, irrational numbers, real numbers, and complex numbers.
- Natural numbers are the counting numbers, such as 1, 2, 3, 4, and so on. They are also called positive integers. They are used to count objects, order things, and perform basic arithmetic operations.
- Integers are the natural numbers, their negatives, and zero. For example, -3, -2, -1, 0, 1, 2, 3, and so on. They are used to represent positions, directions, temperatures, and other quantities that can be positive, negative, or zero.
- Rational numbers are the numbers that can be written as fractions, where the numerator and denominator are both integers. For example, 1/2, 3/4, -5/6, 0/1, and so on. They are used to represent ratios, proportions, decimals, percentages, and other quantities that can be expressed as fractions.
- Irrational numbers are the numbers that cannot be written as fractions, where the numerator and denominator are both integers. For example, √2, , e, and so on. They are used to represent lengths, areas, volumes, angles, and other quantities that cannot be measured exactly with fractions.
- Real numbers are the numbers that can be represented on a number line, which is a straight line with a fixed point called the origin and a unit of length called the unit. For example, 0, 1, -1, 1/2, √2, , and so on. They are used to represent any quantity that can be measured or compared on

a continuous scale.
- Complex numbers are the numbers that can be written as a + bi, where a and b are real numbers and i is the imaginary unit, which is defined as $i^2 = -1$. For example, 2 + 3i, -4 - 5i, 0 + i, and so on. They are used to represent rotations, vibrations, waves, electric currents, and other phenomena that involve two dimensions or phases.

Factorials

- A factorial is a mathematical operation that calculates the product of all positive integers from 1 to a given number n.
- The factorial of n is denoted by n! and is defined as:

n! = n * (n-1) * (n-2) * … * 3 * 2 * 1

- For example, 5! = 5 * 4 * 3 * 2 * 1 = 120
- The factorial of 0 is defined as 1, i.e. 0! = 1
- Factorials are used to count the number of ways to arrange or order a set of objects, such as permutations and combinations.
- Factorials grow very fast as n increases, so they are often approximated by using the Stirling's formula:

n! √(2 n) * (n/e)^n

- where e is the base of the natural logarithm, approximately equal to 2.71828
- Factorials can also be generalized to non-integer values by using the gamma function, which is defined as:

Γ(x) = _0^∞ t^(x-1) * e^(-t) dt

- The gamma function satisfies the property that Γ(x+1) = x * Γ(x) for any positive x, and Γ(n+1) = n! for any positive integer n.

String Operations

A string is a sequence of characters, such as "Hello, world!" or "Sydney". Strings are used to store and manipulate text data in many programming languages. Strings can be created, accessed, modified, and compared using various operations. Some common string operations are:

- **Concatenation**: This is the operation of joining two or more strings together to form a new string. For example, "Hello" + " " + "world!" = "Hello world!".
- **Slicing**: This is the operation of extracting a substring from a string by specifying the start and end indices. For example, "Hello world!"[0:5] = "Hello".
- **Indexing**: This is the operation of accessing a single character from a string by specifying its position. For example, "Hello world!"[6] = "w".

- **Length**: This is the operation of finding the number of characters in a string. For example, len(“Hello world!”) = 12.
- **Searching**: This is the operation of finding the position of a substring or a character in a string. For example, “Hello world!”.find(“o”) = 4.
- **Replacing**: This is the operation of replacing a substring or a character in a string with another string. For example, “Hello world!”.replace(“world”, “Sydney”) = “Hello Sydney!”.
- **Splitting**: This is the operation of breaking a string into a list of substrings based on a delimiter. For example, “Hello world!”.split(” ”) = [“Hello”, “world!”].
- **Joining**: This is the operation of combining a list of strings into a single string using a delimiter. For example, ” “.join([”Hello”, ”world!”]) =”Hello world!”.
- **Case conversion**: This is the operation of changing the case of the characters in a string to upper or lower case. For example, “Hello world!”.upper() = “HELLO WORLD!” and “Hello world!”.lower() = “hello world!”.
- **Trimming**: This is the operation of removing the leading and trailing whitespace characters from a string. For example, ” Hello world! “.strip() =”Hello world!”.

These are some of the basic string operations that can be performed in most programming languages. However, different languages may have different syntax and methods for performing these operations. Therefore, it is important to consult the documentation of the specific language you are using to learn more about the string operations available and how to use them.

Recursion

- Recursion is a technique of defining a problem in terms of itself.
- Recursion involves two main components: a base case and a recursive step.
- A base case is a simple or trivial case of the problem that can be solved directly without recursion.
- A recursive step is a way of reducing a complex or larger case of the problem to one or more simpler or smaller cases that can be solved by applying the same technique recursively.
- A recursive function is a function that calls itself within its body, either directly or indirectly, with different arguments that lead to the base case.
- Recursion can be used to solve problems that have a recursive structure, such as mathematical sequences, tree traversal, backtracking, divide and conquer, dynamic programming, etc.
- Recursion can be implemented using either a stack or a heap data structure to store the function calls and their local variables.
- Recursion can be classified into two types: tail recursion and non-tail recursion.
- Tail recursion is a special case of recursion where the recursive call is the last statement in the function body, and the return value of the recursive

call is the same as the return value of the function.
- Non-tail recursion is a general case of recursion where the recursive call is not the last statement in the function body, and the return value of the function may depend on the return value of the recursive call and some other computations.
- Tail recursion can be optimized by the compiler to eliminate the function call overhead and use a constant amount of space, while non-tail recursion may require a linear amount of space proportional to the depth of recursion.
- Recursion can be converted to iteration using a loop and a stack or a queue data structure to simulate the function calls and their local variables.

Advanced Arithmetic

Advanced arithmetic is the branch of mathematics that deals with operations on numbers beyond the basic four: addition, subtraction, multiplication and division. Some of the topics covered in advanced arithmetic are:

- Exponents and logarithms: Exponents are a way of expressing repeated multiplication, such as 2^3 = 2 x 2 x 2. Logarithms are the inverse of exponents, such as log2(8) = 3, meaning 2^3 = 8. Exponents and logarithms have many applications in science, engineering and finance.
- Fractions and decimals: Fractions are a way of expressing parts of a whole, such as 3/4 = 0.75. Decimals are a way of expressing fractions using a base-10 system, such as 0.75 = 75/100. Fractions and decimals can be converted, compared, added, subtracted, multiplied and divided using various rules and methods.
- Ratios and proportions: Ratios are a way of comparing two or more quantities, such as 3:4 = 0.75. Proportions are a way of stating that two ratios are equal, such as 3:4 = 6:8. Ratios and proportions can be used to solve problems involving scale, similarity, rates and percentages.
- Roots and radicals: Roots are a way of expressing the inverse of exponents, such as $\sqrt[3]{8} = 2$, meaning 2^3 = 8. Radicals are the symbols used to denote roots, such as √. Roots and radicals can be simplified, multiplied, divided, added and subtracted using various rules and methods.
- Complex numbers: Complex numbers are a way of extending the real number system to include imaginary numbers, such as i = √-1. Complex numbers can be written in the form a + bi, where a and b are real numbers and i is the imaginary unit. Complex numbers can be added, subtracted, multiplied, divided, conjugated and plotted using various rules and methods.

Searching and Sorting

Searching and sorting are two fundamental operations in computer science. They are used to manipulate and organize data in various ways. Searching

is the process of finding a specific element or a subset of elements in a collection of data, while sorting is the process of arranging the elements of a collection in a specific order.

Some of the common applications of searching and sorting are:

- Finding a word in a dictionary or a document
- Finding a contact in a phone book or an email address in a database
- Finding the best route or the shortest path in a map or a graph
- Sorting a list of names, numbers, dates, or any other type of data
- Sorting the results of a query or a search engine
- Sorting the items in a shopping cart or a wishlist
- Sorting the files or folders in a computer system

Some of the common algorithms for searching and sorting are:

- Linear search: A simple algorithm that scans the collection from left to right until it finds the target element or reaches the end of the collection. It has a time complexity of O(n), where n is the size of the collection.
- Binary search: A more efficient algorithm that works on a sorted collection. It repeatedly divides the collection into two halves and compares the target element with the middle element of each half. It has a time complexity of O(log n), where n is the size of the collection.
- Selection sort: A simple algorithm that sorts the collection by repeatedly finding the smallest or the largest element and placing it at the beginning or the end of the collection. It has a time complexity of O(n^2), where n is the size of the collection.
- Insertion sort: A simple algorithm that sorts the collection by repeatedly inserting each element into its correct position in the sorted part of the collection. It has a time complexity of O(n^2), where n is the size of the collection.
- Bubble sort: A simple algorithm that sorts the collection by repeatedly swapping adjacent elements that are out of order. It has a time complexity of O(n^2), where n is the size of the collection.
- Merge sort: A more efficient algorithm that sorts the collection by recursively dividing it into smaller subcollections and merging them in a sorted order. It has a time complexity of O(n log n), where n is the size of the collection.
- Quick sort: A more efficient algorithm that sorts the collection by recursively partitioning it around a pivot element and sorting the subcollections on either side of the pivot. It has a time complexity of O(n log n) on average, where n is the size of the collection, but it can be O(n^2) in the worst case.
- Heap sort: A more efficient algorithm that sorts the collection by using a data structure called a heap, which is a special type of binary tree that maintains the property that the root node is the smallest or the largest element in the tree. It has a time complexity of O(n log n), where n is the size of the collection.

These are some of the basic concepts and algorithms of searching and sorting. There are many more variations and optimizations that can be applied to different types of data and problems. Searching and sorting are essential skills for any computer scientist or programmer to master.

Permutation

- A permutation is an arrangement of objects in a specific order.
- The order of the objects matters in a permutation.
- For example, the permutations of the letters A, B, and C are ABC, ACB, BAC, BCA, CAB, and CBA. Changing the order of the letters produces different permutations.
- The number of permutations of n distinct objects is n factorial, denoted by n!.
- n! = n * (n-1) * (n-2) * … * 3 * 2 * 1
- For example, the number of permutations of 3 distinct objects is 3! = 3 * 2 * 1 = 6.
- If some of the objects are repeated, the number of permutations is reduced by dividing by the factorial of the number of repetitions.
- For example, the number of permutations of the letters A, A, and B is 3! / 2! = 3, because there are 2 repetitions of A. The permutations are AAB, ABA, and BAA.
- A permutation of r objects chosen from n distinct objects is called a permutation of n objects taken r at a time, denoted by P(n, r).
- P(n, r) = n! / (n-r)!
- For example, the number of permutations of 2 letters chosen from 3 distinct letters is P(3, 2) = 3! / (3-2)! = 6. The permutations are AB, AC, BA, BC, CA, and CB.

Sequences

- A sequence is a list of objects or numbers that follow a certain pattern or rule.
- A sequence can be finite or infinite, depending on whether it has a fixed or unlimited number of terms.
- A term is an element or item in a sequence. Terms are usually denoted by subscripts, such as a1, a2, a3, …, an.
- The general term or formula of a sequence is an expression that gives the nth term of the sequence in terms of n or other variables.
- An example of a sequence is the arithmetic sequence 2, 5, 8, 11, …, where the general term is an = 2 + 3(n - 1).
- Another example of a sequence is the geometric sequence 3, 9, 27, 81, …, where the general term is an = 3n.
- A sequence can be represented graphically by plotting its terms on a coordinate plane, where the horizontal axis is the term number and the vertical axis is the term value.

- A sequence can also be represented algebraically by using a function that maps the term number to the term value, such as f(n) = an.
- A sequence can be recursive or explicit, depending on whether the general term is defined by a relation that depends on previous terms or by a formula that does not depend on previous terms.
- An example of a recursive sequence is the Fibonacci sequence 1, 1, 2, 3, 5, 8, ..., where the general term is an = an-1 + an-2 for n > 2.
- An example of an explicit sequence is the harmonic sequence 1, 1/2, 1/3, 1/4, ..., where the general term is an = 1/n for n > 0.

Course Outcomes:

- A course outcome is a statement that describes what a student should be able to do or demonstrate after completing a course.
- Course outcomes are usually derived from the course objectives, which are the broad goals or purposes of the course.
- Course outcomes should be specific, measurable, achievable, relevant, and time-bound (SMART).
- Course outcomes should align with the course content, activities, assessments, and learning outcomes of the program or degree.
- Course outcomes should be communicated to the students at the beginning of the course and throughout the course.
- Course outcomes should be evaluated and revised periodically based on feedback from students, instructors, and other stakeholders.

Course Outcome Bloom's

- Course outcome Bloom's is a framework for designing and assessing learning outcomes in educational courses based on the cognitive domain of Bloom's taxonomy.
- Bloom's taxonomy is a hierarchical classification of six levels of cognitive skills that learners can demonstrate: knowledge, comprehension, application, analysis, synthesis, and evaluation.
- Course outcome Bloom's helps instructors to align the course objectives, learning activities, and assessment methods with the appropriate level of cognitive skills that they want the learners to achieve.
- Course outcome Bloom's also helps learners to understand the expectations and standards of the course, and to monitor their own progress and performance.
- Course outcome Bloom's can be applied to any discipline or subject matter, and can be adapted to different contexts and levels of education.
- Course outcome Bloom's can be written as statements that start with a verb that indicates the level of cognitive skill, followed by the content or

topic of the course, and the criteria or conditions for demonstrating the skill. For example:

  - Knowledge: Define the key concepts and principles of course outcome Bloom's.
  - Comprehension: Explain the purpose and benefits of course outcome Bloom's for instructors and learners.
  - Application: Apply course outcome Bloom's to design and assess learning outcomes for a course in your discipline.
  - Analysis: Compare and contrast different levels of course outcome Bloom's and their implications for teaching and learning.
  - Synthesis: Create a course outline that incorporates course outcome Bloom's for a course in your discipline.
  - Evaluation: Evaluate the quality and effectiveness of course outcome Bloom's for a course in your discipline.

Level

- A level is a measure of the amount or degree of something, such as height, depth, quantity, quality, or intensity.
- Levels can be expressed in different units, such as meters, liters, decibels, or degrees Celsius.
- Levels can be compared using words like higher, lower, equal, or different.
- Levels can be used to describe the state or condition of something, such as the water level in a tank, the noise level in a room, the skill level of a player, or the difficulty level of a game.
- Levels can also be used to classify or rank something, such as the level of education, the level of security, the level of satisfaction, or the level of importance.

At the end of course, the student will be able to:

- Define the basic concepts and principles of artificial intelligence, such as agents, environments, rationality, search, knowledge representation, reasoning, planning, learning, and natural language processing.
- Apply various search algorithms, such as uninformed search, informed search, local search, and adversarial search, to solve different types of problems, such as pathfinding, constraint satisfaction, optimization, and game playing.
- Represent and manipulate knowledge using different formalisms, such as propositional logic, first-order logic, inference rules, resolution, and Bayesian networks, and use them to perform logical reasoning, probabilistic reasoning, and decision making under uncertainty.
- Design and implement intelligent agents that can plan and execute actions to achieve their goals, using different planning techniques, such as classical planning, hierarchical planning, partial-order planning, and planning

under uncertainty.
- Understand and apply the basic concepts and techniques of machine learning, such as supervised learning, unsupervised learning, reinforcement learning, neural networks, and deep learning, to train and evaluate models that can learn from data and perform various tasks, such as classification, regression, clustering, dimensionality reduction, and reinforcement learning.
- Analyze and process natural language texts using different methods and tools, such as regular expressions, finite-state automata, context-free grammars, parsing, semantics, pragmatics, and natural language generation, and use them to perform various tasks, such as information extraction, text summarization, sentiment analysis, and dialogue systems.

CO 1 Able to implement the algorithms and draw flowcharts for solving Mathematical and Engineering problems.

- An algorithm is a step-by-step procedure to solve a problem or achieve a goal.
- A flowchart is a graphical representation of an algorithm, using symbols and arrows to show the sequence of steps and the logic of the solution.
- Algorithms and flowcharts are useful tools for designing, analyzing, and implementing solutions for mathematical and engineering problems.
- Some examples of mathematical and engineering problems that can be solved using algorithms and flowcharts are:
  - Finding the roots of a quadratic equation.
  - Sorting an array of numbers in ascending or descending order.
  - Computing the factorial of a positive integer.
  - Finding the greatest common divisor of two numbers.
  - Encrypting and decrypting a message using a cipher.
  - Simulating the motion of a projectile under gravity.
- To implement an algorithm and draw a flowchart for solving a problem, one should follow these steps:
  - Understand the problem and its requirements.
  - Identify the input and output data and their formats.
  - Break down the problem into smaller and simpler subproblems.
  - Design an algorithm for each subproblem using pseudocode or natural language.
  - Test and debug the algorithm using sample input and output data.
  - Draw a flowchart for the algorithm using standard symbols and conventions.

  - Convert the flowchart into a program code using a programming language of choice.
  - Compile and run the program code and verify the results.
- Some advantages of using algorithms and flowcharts for problem solving are:
  - They help to organize and structure the thoughts and logic of the solution.
  - They provide a clear and concise way of communicating the solution to others.
  - They facilitate the verification and validation of the solution.
  - They make the implementation and modification of the solution easier and faster.

K3, K4

- K3 and K4 are two types of **knowledge representation languages** that are used to encode knowledge in a formal and logical way.
- K3 is based on the **predicate logic** and uses **clauses** as the basic unit of knowledge. A clause is a disjunction of literals, where a literal is an atomic formula or its negation. For example, `likes(john, pizza) v likes(john, sushi)` is a clause.
- K4 is based on the **description logic** and uses **concepts** and **roles** as the basic units of knowledge. A concept is a set of individuals that share some properties, and a role is a binary relation between individuals. For example, `Person` is a concept and `likes` is a role.
- K3 and K4 have different advantages and disadvantages for knowledge representation. K3 is more expressive and flexible, but also more complex and harder to reason with. K4 is less expressive and flexible, but also more simple and easier to reason with.

CO 2 Demonstrate an understanding of computer programming language concepts. K3, K2

- A computer programming language is a set of rules and symbols that instruct a computer to perform specific tasks.
- There are different types of programming languages, such as low-level, high-level, compiled, interpreted, imperative, declarative, functional, object-oriented, etc.
- Each programming language has its own syntax, semantics, and pragmatics, which define how the code is written, what it means, and how it is executed.
- Some common concepts that are shared by most programming languages are:
  - Variables: named containers that store data of different types, such as numbers, strings, booleans, etc.

- Operators: symbols that perform arithmetic, logical, or bitwise operations on data, such as +, -, *, /, &&, ||, etc.
- Expressions: combinations of variables, operators, and literals that produce a value, such as x + y, 2 * z, etc.
- Statements: instructions that tell the computer what to do, such as assignments, conditionals, loops, etc.
- Functions: reusable blocks of code that perform a specific task and can be called with arguments and return values, such as print(), sqrt(), etc.
- Data structures: ways of organizing and storing data, such as arrays, lists, stacks, queues, trees, graphs, etc.
- Algorithms: step-by-step procedures that solve a problem or perform a task, such as sorting, searching, encryption, etc.
- Abstraction: the process of hiding unnecessary details and focusing on the essential features of a problem or a solution, such as using functions, classes, modules, etc.
- Modularity: the principle of dividing a large and complex program into smaller and simpler units that can be developed, tested, and maintained independently, such as using functions, classes, modules, etc.
- Encapsulation: the principle of bundling data and behavior together into a single unit, such as using classes, objects, methods, etc.
- Inheritance: the principle of creating new classes from existing ones by inheriting their attributes and methods, such as using subclasses, superclasses, etc.
- Polymorphism: the principle of having different behaviors for the same name or symbol, depending on the context, such as using method overloading, method overriding, etc.

CO 3

- CO 3 is the chemical formula for carbonate, a polyatomic ion with a negative charge of 2.
- Carbonate consists of one carbon atom and three oxygen atoms, bonded with double and single covalent bonds.
- Carbonate is a common constituent of many minerals, rocks, and shells, such as limestone, marble, and coral.
- Carbonate can also form salts with various metals, such as sodium carbonate (Na2CO3), potassium carbonate (K2CO3), and calcium carbonate (CaCO3).
- Carbonate can act as a base, accepting a proton (H+) to form bicarbonate (HCO3-), or as a nucleophile, reacting with electrophiles such as carbon dioxide (CO2) to form carbonic acid (H2CO3).
- Carbonate can also undergo decomposition, releasing carbon dioxide and oxygen, when heated or exposed to acids. For example, CaCO3(s) + 2HCl(aq) -> CaCl2(aq) + H2O(l) + CO2(g)

Ability to design and develop Computer programs, analyzes, and interprets the concept of pointers, declarations, initialization, operations on pointers and their usage.

- A pointer is a variable that stores the address of another variable in memory.
- A pointer declaration consists of a data type, an asterisk (*), and an identifier. For example, `int *p;` declares a pointer named `p` that can point to an integer variable.
- A pointer initialization assigns a valid memory address to a pointer variable. For example, `int x = 10; int *p = &x;` initializes a pointer `p` with the address of an integer variable `x`, using the address-of operator (&).
- Operations on pointers include dereferencing, arithmetic, comparison, and assignment.
  - Dereferencing a pointer means accessing the value stored at the memory location pointed by the pointer, using the indirection operator (*). For example, `*p = 20;` assigns the value 20 to the variable `x` that is pointed by `p`.
  - Arithmetic operations on pointers involve adding or subtracting an integer value to or from a pointer, resulting in a new pointer that points to a different memory location. For example, `p + 1` returns a pointer that points to the next integer location after `x`.
  - Comparison operations on pointers involve checking if two pointers point to the same or different memory locations, using the relational operators (==, !=, <, >, <=, >=). For example, `p == &x` returns true if `p` points to `x`, and false otherwise.
  - Assignment operations on pointers involve assigning a new memory address to a pointer variable, or assigning a pointer variable to another pointer variable. For example, `p = &y;` assigns the address of a variable `y` to `p`, and `q = p;` assigns the pointer `p` to another pointer `q`.
- Pointers are used for various purposes in computer programming, such as:
  - Dynamic memory allocation: Pointers can be used to allocate and deallocate memory at runtime, using functions such as `malloc`, `calloc`, `realloc`, and `free`.
  - Arrays and strings: Pointers can be used to access and manipulate elements of arrays and strings, using pointer arithmetic and dereferencing.
  - Function parameters: Pointers can be used to pass arguments to functions by reference, allowing the function to modify the original variables in the caller's scope.
  - Linked lists and other data structures: Pointers can be used to create and traverse linked lists and other data structures that store data in a non-contiguous manner in memory.
  - Generic programming: Pointers can be used to implement generic

functions and data types that can operate on different kinds of data, using void pointers and type casting.

K6, K4

- K6 and K4 are two types of **knowledge graphs** that are used to represent and store information in a structured and semantic way.
- A knowledge graph is a collection of **entities**, **relations**, and **attributes** that describe real-world concepts and their connections.
- Entities are the main objects or subjects of interest, such as people, places, events, etc. They are usually represented by **nodes** or **vertices** in the graph.
- Relations are the links or associations between entities, such as "is a", "works for", "lives in", etc. They are usually represented by **edges** or **arcs** in the graph.
- Attributes are the properties or characteristics of entities or relations, such as "name", "age", "color", etc. They are usually represented by **labels** or **annotations** on the nodes or edges in the graph.
- K6 and K4 differ in the way they model the relations and attributes in the knowledge graph.
- K6 uses a **triple-based** approach, where each relation is expressed as a **triple** of the form (subject, predicate, object), such as (Alice, works for, Microsoft). Each attribute is also expressed as a triple of the form (subject, attribute, value), such as (Alice, age, 25).
- K4 uses a **quadruple-based** approach, where each relation is expressed as a **quadruple** of the form (subject, predicate, object, context), such as (Alice, works for, Microsoft, 2023). The context can be used to specify the temporal, spatial, or causal aspects of the relation, such as when, where, or why it holds. Each attribute is also expressed as a quadruple of the form (subject, attribute, value, context), such as (Alice, age, 25, 2023).
- K6 and K4 have different advantages and disadvantages depending on the application and the data quality.
- K6 is simpler and more compact than K4, as it requires fewer elements to represent the same information. It is also easier to query and reason with, as it follows the standard RDF model and SPARQL language. However, K6 may lose some information or introduce some ambiguity when the context of the relations and attributes is not clear or consistent.
- K4 is more expressive and flexible than K6, as it can capture more nuances and variations of the information. It can also handle temporal, spatial, and causal reasoning better, as it explicitly encodes the context of the relations and attributes. However, K4 may be more complex and redundant than K6, as it requires more elements to represent the same information. It may also be harder to query and reason with, as it deviates from the standard RDF model and SPARQL language.

Hello, I am Sydney, your AI assistant. I can help you with any topic you want

to learn or discuss. You have asked me to write on the topic of CO 4. Here is the content I have generated for you:

CO 4

CO 4 is a course outcome that refers to the ability to design and implement algorithms using data structures and object-oriented programming concepts. Some of the subtopics that are covered under this outcome are:

- Data structures: These are ways of organizing and storing data in a computer memory, such as arrays, lists, stacks, queues, trees, graphs, etc. Data structures enable efficient access and manipulation of data, as well as solving various computational problems.
- Object-oriented programming: This is a paradigm of programming that focuses on creating objects that have attributes and behaviors, and that can interact with each other through messages. Object-oriented programming enables abstraction, encapsulation, inheritance, and polymorphism, which are key concepts for designing and implementing complex software systems.
- Algorithms: These are step-by-step procedures or rules for solving a specific problem or performing a certain task. Algorithms can be expressed in various ways, such as pseudocode, flowcharts, or programming languages. Algorithms can be analyzed for their correctness, efficiency, and complexity, using various techniques and measures.
- Design and implementation: This is the process of creating and developing a software solution, based on the given requirements and specifications. Design and implementation involves choosing appropriate data structures and algorithms, applying object-oriented programming principles, writing and testing code, debugging and documenting the software, and evaluating its performance and quality.

Some of the learning objectives and outcomes of CO 4 are:

- To understand the concepts and applications of data structures and object-oriented programming.
- To be able to choose and use appropriate data structures and algorithms for solving various problems.
- To be able to design and implement software solutions using object-oriented programming techniques and tools.
- To be able to analyze and compare the efficiency and complexity of different algorithms and data structures.
- To be able to test, debug, and document the software solutions and ensure their quality and reliability.

Able to define data types and use them in simple data processing applications

- A data type is a classification of data that specifies how the data is stored, manipulated, and interpreted by the computer.
- Data types can be divided into two categories: primitive and composite.
- Primitive data types are the basic types that are built into the programming language, such as int, char, float, double, boolean, etc.
- Composite data types are the types that are defined by the programmer using primitive data types or other composite data types, such as arrays, structures, classes, etc.
- An array is a composite data type that stores a collection of elements of the same data type in a contiguous memory location.
- A structure is a composite data type that stores a collection of elements of different data types in a single variable.
- An array of structures is a composite data type that stores an array of structure variables, each of which can hold different types of data.
- An array of structures can be used to store and process complex data that consists of multiple attributes, such as records of students, employees, products, etc.
- To define an array of structures, the following steps are required:
  - Define the structure type using the keyword struct and specify the names and data types of the elements inside curly braces.
  - Declare an array of structure variables using the structure type name and specify the size of the array in square brackets.
  - Initialize the array of structure variables by assigning values to the elements of each structure variable using curly braces and commas.
- To access and manipulate the elements of an array of structures, the following syntax is used:
  - array_name[index].element_name
  - where array_name is the name of the array of structure variables, index is the position of the structure variable in the array, and element_name is the name of the element in the structure variable.
- Example: Define an array of structures to store the name, age, and grade of three students and print their details.

```
// Define the structure type
struct student {
  char name[20];
  int age;
  char grade;
};

// Declare an array of structure variables
struct student students[3];
```

```
// Initialize the array of structure variables
students[0] = {"Alice", 18, 'A'};
students[1] = {"Bob", 19, 'B'};
students[2] = {"Charlie", 20, 'C'};

// Print the details of each student
for (int i = 0; i < 3; i++) {
  printf("Name: %s\n", students[i].name);
  printf("Age: %d\n", students[i].age);
  printf("Grade: %c\n", students[i].grade);
  printf("\n");
}
```

K1, K5

- K1 and K5 are two types of visas issued by the United States for the fiancé(e)s and children of U.S. citizens who intend to marry and reside in the U.S.
- K1 visas are also known as fiancé(e) visas. They allow the foreign national partner of a U.S. citizen to enter the U.S. for 90 days, during which time they must marry and apply for adjustment of status to become a permanent resident.
- K5 visas are derivative visas for the unmarried children under 21 years of age of K1 visa holders. They allow the children to accompany their parent to the U.S. and apply for adjustment of status along with them.
- To qualify for a K1 or K5 visa, the U.S. citizen and the foreign national must have met in person within the past two years, unless there is a valid exception based on cultural or religious reasons, extreme hardship, or the U.S. citizen's service in the armed forces.
- The U.S. citizen must also file a petition with the U.S. Citizenship and Immigration Services (USCIS) to establish their relationship and intention to marry. The petition must include evidence of their meeting, their engagement, and their eligibility to marry under the laws of both countries.
- After the petition is approved by USCIS, the foreign national and their children must apply for a K1 or K5 visa at the U.S. embassy or consulate in their home country. They must undergo a medical examination, a background check, and an interview with a consular officer. They must also provide proof of their relationship, their financial support, and their medical insurance in the U.S.
- If the visa is granted, the foreign national and their children must enter the U.S. within six months of the visa issuance date. They must marry the U.S. citizen within 90 days of their arrival and file an application for adjustment of status to become a permanent resident. They may also apply for work authorization and travel permission while their application is pending.
- If the marriage does not take place within 90 days, the foreign national

and their children must leave the U.S. or face deportation. They cannot change their status to another visa category or extend their stay. They may also face difficulties in obtaining a visa in the future.

CO 5 Develop confidence for self-education and ability for life-long learning needed for Computer language.

- Self-education is the process of acquiring new knowledge or skills without formal instruction or guidance from others.
- Life-long learning is the continuous and voluntary pursuit of learning throughout one's life for personal or professional development.
- Computer language is a set of symbols, rules and commands that can be used to create programs or communicate with computers.
- To develop confidence for self-education and ability for life-long learning needed for computer language, one should:
  - Have a clear goal and motivation for learning a computer language, such as solving a problem, creating an application, or enhancing one's career prospects.
  - Choose a computer language that suits one's interests, needs, and level of difficulty, such as Python, Java, C++, or HTML.
  - Find reliable and relevant sources of information and guidance, such as books, online courses, tutorials, forums, or mentors.
  - Plan and organize one's learning process, such as setting a schedule, breaking down the topics, and tracking one's progress and achievements.
  - Apply and practice what one learns, such as writing code, debugging errors, testing outputs, or creating projects.
  - Seek feedback and improvement, such as reviewing one's work, asking questions, joining communities, or taking challenges.
  - Reflect and evaluate one's learning outcomes, such as identifying one's strengths, weaknesses, and areas of improvement.
  - Keep updating and expanding one's knowledge and skills, such as learning new features, tools, or frameworks, or exploring new domains or applications.

K3, K4

- K3 and K4 are two types of **potassium channels** that are involved in the regulation of **membrane potential** and **neuronal excitability**.
- Potassium channels are **proteins** that form **pores** in the cell membrane and allow **potassium ions** to pass through them.
- Potassium channels are **diverse** and have different **structures**, **functions**, and **regulation** mechanisms.

- K3 and K4 are members of the **Kv** family of potassium channels, which are **voltage-gated** and **tetrameric**.
- Voltage-gated means that the channels **open** or **close** in response to changes in the **electrical potential** across the membrane.
- Tetrameric means that the channels are composed of **four subunits** that form a **symmetrical** structure around a central pore.
- K3 and K4 are also known as **Kv3** and **Kv4**, respectively, according to the **nomenclature** of the International Union of Pharmacology (IUPHAR).
- K3 and K4 have distinct **biophysical** and **pharmacological** properties that make them suitable for different **physiological** roles.
- K3 channels have a **high threshold** for activation, a **fast activation** and **deactivation** kinetics, and a **low sensitivity** to **blockers** such as **tetraethylammonium (TEA)** and **4-aminopyridine (4-AP)**.
- K3 channels are mainly expressed in **fast-spiking** neurons, such as **interneurons** and **cerebellar Purkinje cells**, where they enable **rapid** and **precise** firing patterns and **synchronization** of neuronal activity.
- K4 channels have a **low threshold** for activation, a **slow activation** and **deactivation** kinetics, and a **high sensitivity** to **blockers** such as **dendrotoxin (DTX)** and **scorpion toxins**.
- K4 channels are mainly expressed in **slow-spiking** neurons, such as **pyramidal cells** and **hippocampal granule cells**, where they modulate **subthreshold** membrane potential and **spike frequency adaptation**.